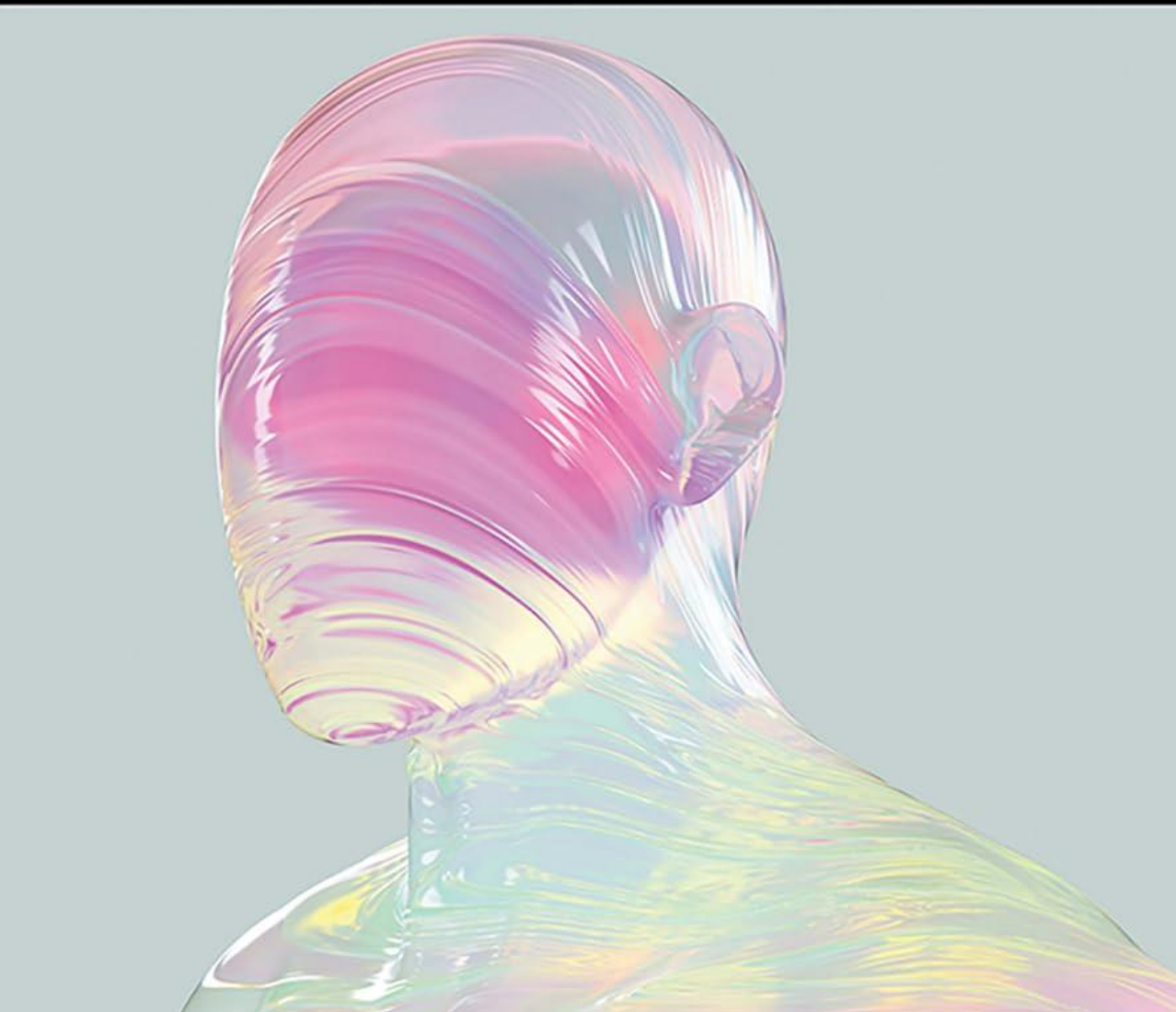
HANDBOOK OF Digital and Computational Research Methods

In the Social Sciences and Humanities

Edited by
ANDERS KOED MADSEN • ANDERS KRISTIAN MUNK

EE Elgar



© Anders Koed Madsen and Anders Kristian Munk 2026

Cover image: Maxim Berg on Unsplash.

Every effort has been made to trace all the copyright holders but if any have been inadvertently overlooked please notify the publisher.

With the exception of any material published open access under a Creative Commons license (see www.elgaronline.com), all rights are reserved and no part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical or photocopying, recording, or otherwise without the prior permission of the publisher.



Chapter 11 is available for free as Open Access from the individual product page at www.elgaronline.com under a Creative Commons Attribution 4.0 International (https://creativecommons.org/licenses/by/4.0/) license.

Published by

Edward Elgar Publishing Limited
The Lypiatts
15 Lansdown Road
Cheltenham
Glos GL50 2JA
UK

Edward Elgar Publishing, Inc.
William Pratt House
9 Dewey Court
Northampton
Massachusetts 01060
USA

Authorised representative in the EU for GPSR queries only: Easy Access System Europe – Mustamäe tee 50, 10621 Tallinn, Estonia, gpsr.requests@easproject.com

A catalogue record for this book is available from the British Library

A LCCN number for this book is available from the Library of Congress

This book is available electronically in the Elgaronline Business subject collection
https://doi.org/10.4337/9781802208993

ISBN 978 1 80220 898 6 (cased)
ISBN 978 1 80220 899 3 (eBook)
ISBN 978 1 0353 8071 8 (ePub)

Contents

List of figures viii
List of editors and contributors x

1 Heterodox research practices in digital and computational SSH (Introduction to the *Handbook of Digital and Computational Research Methods*) 1
Anders Koed Madsen and Anders Kristian Munk

PART I DISCIPLINARY PERSPECTIVES

2 Archaeology 19
Iza Romanowska

3 The digital historian's craft 32
Martin Grandjean

4 Computational anthropology 48
Samantha Breslin and Kristoffer Albris

5 Economic sociology: Using agent-based modeling to study financial markets 65
Christian Borch and Nicholas Skar-Gislinge

6 Four guiding principles for rethinking organizational charts 80
Dario Rodighiero

7 The digital turn in food studies 95
Tanja Schneider

8 Combining qualitative and quantitative methods in behavioural psychology for complex human-environment systems 112
Jens Koed Madsen

9 Situating digital data in media and communication research 129
Sharon Meraz

10 Objectivity in/through partial perspectives 150
Laura K. Nelson

PART II EPISTEMIC TROUBLE

11 Bias in machine learning as a multi-dimensional problem 160
Mascha Kurpicz-Briki

12 Online groundedness 177
Richard Rogers

13 Theoretical heuristics in digital social research 190
Hjalmar Bang Carlsen

14 Beyond the quant and qual divide 201
David Moats

15 The vanishing micro/macro divide and the politics of computational interactionism 221
Tommaso Venturini

16 Gatekeeping: The case of digital archives 232
Johan Heinsen

17 Representation and visualization 244
Michele Mauri

18 Mapping with care: Data feminist principles for participatory visual practices 259
Sabine Niederer and Gabriele Colombo

19 Privacy in digital and computational SSH 273
Nicholas Proferes

PART III MAKING THE INSTRUMENTS OF SOCIAL SCIENCES AND HUMANITIES

Introduction to Part III 290

20 Toolmaking is science by other means 294
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

21 Meet the DCSSH toolmakers 307
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

22 Why do DCSSH toolmakers make tools? 314
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

23 The lifecyle of DC tools for SSH 325
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

24 Valuing toolmaking in academia 339
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

25 In defense of the researcher-toolmaker figure 349
Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Index 352

Introduction to Part III

We, the authors of the chapters in this section, make tools used in the digital and computational social sciences and humanities (DCSSH). Some of the more well known are *Gephi*[1] (Bastian et al., 2009) for network visualization; *CitNetExplorer*[2] and *VOSviewer*[3] (Van Eck and Waltman, 2017) for bibliometrics; *DaCy*[4] (Enevoldsen et al., 2021) and *Talisman*[5] (Plique, 2020) for natural language processing; *4CAT*[6] (Peeters and Hagen, 2022), *DMI-TCAT*[7] (Borra and Rieder, 2014), *Contropedia*[8] (Borra et al., 2015), *Hyphe*[9] (Jacomy et al., 2016), *Minet*[10] (Plique et al., 2025), *Netvizz*[11] (Rieder, 2013), and the *YouTube Data Tools*[12] (Rieder, 2015) for social media research; and *Prompt Compass*[13] (Borra, 2023b) for applying large language models to SSH.

It is in the nature of a tool to be *already there*, taken for granted. When scholars in the social sciences or humanities become aware of a tool that might help them, they may receive it under a series of assumptions such as: this tool has been made by professional toolmakers; it has been designed for scholars whose needs are similar to mine; its promoted use is the purpose of its design; it comes with a method that should work for me like it does for others; its popularity means that it has been validated by the research community; it must be funded and maintained by people whose job it is to maintain it; and it will last. Any of these things are far from always the case, and understanding why may help you engage better with your scientific apparatus, or even contribute to improving it.

Throughout the chapters in this section, we aim to challenge the assumptions listed above, because we experience how precarious such tools can be. We want to explain why a tool may very well have been brought into existence accidentally by people who just wanted to meet their own niche goals, did not think of it as a tool (let alone a popular one), yet got stuck with it when it circulated to strangers who repurposed and reappropriated it, enrolling it into their own methods, shaping the culture surrounding it through artifacts such as tutorials, academic papers, or plugins, accentuating a disconnection between the users and the toolmaker. Alternatively, a tool may have been created and be maintained for a non-academic public by professional developers who are not researchers and never get to meet the users (e.g. *OpenRefine*;[14] Ham, 2013). Toolmaking is a diverse landscape, even just looking at applications to the DCSSH. Even a popular tool may well become unfunded, unmaintained, abandoned in a precarious state by its inadvertent creator, or cut off from its source of data, until it breaks and gets abandoned by its users as well. Acknowledging where the scholarly technology comes from will help us understand why and how it was shaped.

That being said, most digital and computational tools for the social sciences and humanities (which we will abbreviate as "DC tools for SSH") do not have a tragic destiny; they just live a seemingly peaceful life where change is infrequent and undramatic. Each scientific instrument has a unique combination of traits, and its situation changes over its lifecycle as it gets redeveloped, becomes popular, gets funded, passes from one group to another, and so on. Nevertheless, almost any tool features one troublesome trait: regardless of the amount of

methodological commitment baked into it, its design was not aimed at enacting the state-of-the-art method. Not only because the state of the art almost never boils down to one consensual method, but first and foremost because the toolmaker's motivation is elsewhere.

Understanding the origins, trajectories, and limitations of DC tools for SSH is not just an academic exercise—it directly impacts how scholars conduct their research. Tools are never static; they evolve, break, get discontinued, or resist adaptation, sometimes in ways that fundamentally shape the kinds of questions scholars can ask and answer. Engaging with toolmakers and their processes allows researchers to better navigate these changes, whether by influencing tool development, troubleshooting issues, or even creating custom solutions. It is also an invitation to recognize toolmakers as vital collaborators in the research ecosystem, or perhaps even to become a toolmaker oneself. Moreover, tools often serve as vehicles for scholarly impact: by mastering or contributing to the development of a tool, scholars can find themselves embedded in new research networks and invited into collaborative projects. Understanding the precarious yet powerful role of tools in DCSSH is key to both navigating and shaping the future of our research, which we propose over a six-chapter journey.

ACKNOWLEDGEMENTS

The chapters in Part III follow from a panel about toolmakers in the social sciences and humanities convened by Aalborg University's MASSHINE center during Fall 2023. The panel included all the authors of this piece plus Antonin Delpeuch, maintainer of *OpenRefine*, whom we warmly thank for his ideas. As we, the authors, come from an international and diverse set of institutions, spanning humanities, social sciences, media studies, and scientometrics, the observations and remarks written here do not reflect the experiences of one individual or at a specific university or country, but can be seen as broader and endemic to the field of digital and computational humanities and social sciences. To further drive that point home, we collected and extensively referenced work by other toolmakers inside and outside those fields.

We will use key quotes from a long interview with Geoffrey Rockwell by Jacomy in 2023, as a common thread to the many facets we address. Geoffrey is one of the toolmakers (with the late Stéfan Sinclair) behind *Voyant*, an online platform to explore texts hermeneutically in the digital humanities. We use his experiences with *Voyant* to illustrate some of our points.

NOTES

1. *Gephi* is an open-source software platform for visualizing and analyzing complex networks and graph data through interactive exploration, layout algorithms, and statistical metrics.
2. *CitNetExplorer* (Citation Network Explorer) is a software tool for visualizing and analyzing citation networks of scientific publications, enabling researchers to explore how academic works are connected through references over time.
3. *VOSviewer* is a software tool for constructing and visualizing bibliometric networks, specializing in displaying publication, author, and keyword relationships through distance-based mapping techniques.

4. *DaCy* is a Danish-language Natural Language Processing pipeline built on *SpaCy*, offering high-performance text analysis capabilities specifically optimized for processing Danish texts.
5. *Talisman* is a JavaScript library offering a comprehensive collection of text analysis functions and string similarity metrics, designed for fuzzy matching, pattern detection, and text comparison tasks.
6. *4CAT* (Capture and Analysis Toolkit) is a web-based research tool that captures, processes, and analyzes social media data and web content.
7. *DMI-TCAT* (Digital Methods Initiative - Twitter Capture and Analysis Toolset) is an open-source research tool designed for collecting, archiving, and analyzing Twitter data through advanced querying and visualization capabilities.
8. *Contropedia* is a web-based analytical tool that visualizes and analyzes the evolution of controversies in Wikipedia articles by tracking edit histories, discussions, and content disputes.
9. *Hyphe* is a web-crawling and corpus-building tool designed for SSH, enabling users to collect, categorize, and analyze web data from customized networks of websites.
10. *Minet* is a versatile command-line tool for digital investigations, enabling the efficient collection, extraction, and processing of web data from various sources, including social media, search engines, and websites.
11. *Netvizz* was a tool for extracting data from Facebook for research purposes, allowing users to analyze networks, content, and interactions until it was discontinued due to platform restrictions.
12. *YouTube Data Tools* is a suite of web-based tools for data collection and analysis from YouTube, enabling researchers to gather information on video metadata, comments, channels, and network interactions.
13. *Prompt Compass* is a tool designed to leverage Large Language Models (LLMs) in digital research tasks by providing access to diverse LLMs, offering a library of prompts for digital research, and enabling users to apply these prompts to a series of inputs.
14. *OpenRefine* is a powerful open-source tool for cleaning, transforming, and organizing large datasets, enabling researchers to efficiently explore, edit, and reconcile messy data for analysis.

REFERENCES

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. *Proceedings of the International AAAI Conference on Web and Social Media*, *3*(1), 361–362.

Borra, E. (2023). *Prompt Compass. A Tool for Navigating LLMs and Prompts for Computational Social Science and Digital Humanities Research* [Computer software]. Zenodo.

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. *Aslib Journal of Information Management*, *66*(3), 262–278.

Borra, E., Weltevrede, E., Ciuccarelli, P., Kaltenbrunner, A., Laniado, D., Magni, G., Mauri, M., Rogers, R., & Venturini, T. (2015). Societal controversies in Wikipedia articles. *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems* (pp. 193–196). https://doi.org/10.1145/2702123.2702436

Enevoldsen, K., Hansen, L., & Nielbo, K. (2021). DaCy: A unified framework for Danish NLP. *arXiv preprint: arXiv:2107.05295.*

Ham, K. (2013). OpenRefine (version 2.5). http://openrefine.org. Free, open-source tool for cleaning and transforming data. *Journal of the Medical Library Association*, *101*(3), 233.

Jacomy, M., Girard, P., Ooghe-Tabanou, B., & Venturini, T. (2016). Hyphe, a curation-oriented approach to web crawling for the social sciences. *Proceedings of the International AAAI Conference on Web and Social Media*, *10*(1), 595–598.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. *Computational Communication Research*, *4*(2), 571–589.

Plique, G. (2020). Talisman: A JavaScript archive of fuzzy matching, information retrieval and record linkage building blocks. *Journal of Open Source Software*, *5*(55), 2405.

Plique, G., Breteau, P., Farjas, J., Théro, H., Descamps, J., Pellé, A., Miguel, L., Pichon, C., Christensen, K,, Pontoire, J., & Mazoyer, B. (2025). Minet, a web mining CLI tool & library for python. (4.1.0). Zenodo. https://doi.org/10.5281/zenodo.15411918

Rieder, B. (2013). Studying Facebook via data extraction: The Netvizz application. *Proceedings of the 5th Annual ACM Web Science Conference,* 346–355.

Rieder, B. (2015). *YouTube Data Tools* [software]. Available at: https://tools.digitalmethods.net/netvizz/youtube/ (accessed 09/08/2024).

Van Eck, N. J., & Waltman, L. (2017). Citation-based clustering of publications using CitNetExplorer and VOSviewer. *Scientometrics*, *111*, 1053–1070.

20. Toolmaking is science by other means

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

Toolmaking "is never merely technical" (Rieder et al., 2024). Tooling up SSH never boils down to saving costs and time; it also reconfigures the scholar's practices. Using a tool modifies your agency in deeper ways than just adding a temporary ability. To make new things visible, the tool hides others; by making some methodological paths easier, it makes others comparatively harder; and ultimately, the tool changes your goals. As the popular saying goes, "it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail" (Maslow, 1966, p. 15).

The equipped SSH scholar needs to understand how the apparatus they use impacts the nature of their scholarly activity. This requires acknowledging that tool production is not purely technical, but academically loaded, as we will develop in this chapter. *How* it is loaded depends on each tool, but *that* it is loaded will not come as a surprise to the reader of this handbook.

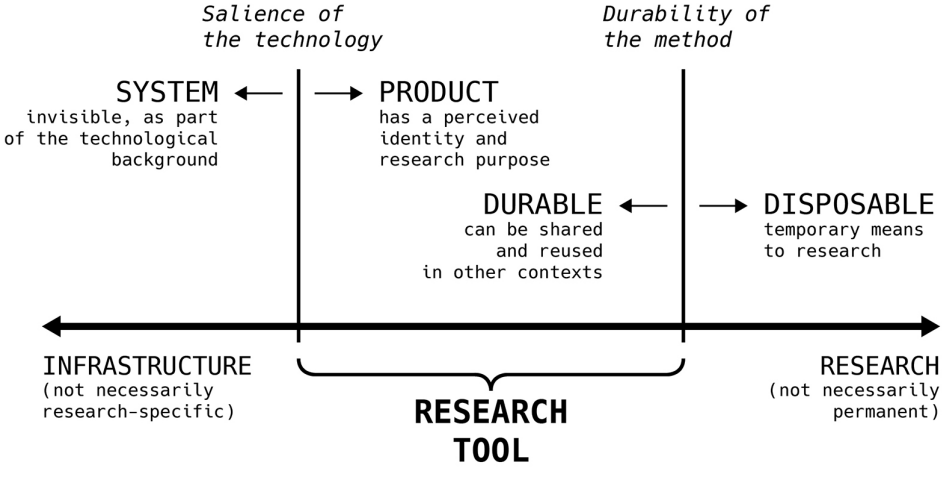
More surprising, however, may be the fact that many DC tools for SSH are not even made for you (as a user); at least not initially. They are academically loaded because they emanate from academic practices, as natural extensions of theories and methods ("natural" from the toolmaker's perspective). This will be our starting point to understand, in this chapter, the specific circumstances under which "tools bear theory" (Rockwell, 2023; Rockwell & Sinclair, 2016).

By painting DCSSH toolmaking upfront as a form of science, we aim to help the SSH scholar look at their DC apparatus differently, attending to what it needs from them as much as what they need from it, and get a better use out of it.

WHAT IS A DC TOOL FOR SSH?

The tools used in the DCSSH are *scientific* instruments; they belong to a large family of technological artifacts—not necessarily computerized—enrolled in the production of knowledge—not necessarily within the SSH—that includes telescopes, microscopes, weighing scales, seismographs, cloud chambers, MRI machines, satellite imagery, and many more. "Instrument" adds a relevant nuance to the word "tool," even though we will often use the latter for brevity: while the purpose of an everyday tool is often to intervene on matter (hammer, saw, soldering iron …), the purpose of a scientific instrument is often, in a broad sense, to make a measurement. Thus, the DC tools for SSH are a subset of scientific instruments that SSH scholars use to facilitate their production of knowledge in a digital and/or computational setting by harvesting data, processing it, and ultimately producing information (text, numbers, visualizations …) that derives from it.

The tools we use in SSH therefore have a double affiliation. As tools, they belong to the series of technological objects that support us in our academic endeavors, which also includes many infrastructures (internet access, library services, cloud storage, IT maintenance …). And as scientific objects, they belong to the series of methodological means through which we shape our academic production. Yet infrastructures and methodology rarely overlap, as we expect a division of labor between the engineers who have the competencies to maintain infrastructures and the academics in charge of the scientific dimension of their methods. Besides, most of the infrastructures used in an academic setting are not specific to science. The double nature of the research tools bridges a divide that is deeply ingrained in universities. We will look at that divide more closely in the next chapter by presenting the figure of the toolmaker. For now, let us simply remark that our research tools live at the intersection of infrastructure and research, which makes them doubly special (Figure 20.1). First, as technological objects, they have the specificity of carrying a scientific purpose, which prevents them from blending into the background of our technological practice. We need to be able to name and discuss our scientific instruments because they are methodologically loaded. Second, as methods, they have the specificity of being durable enough to be reused out of the box. The SSH are rich in methods that need the researcher's active participation to be deployed, but the method implemented by an algorithm does not need such active participation; it can be executed by anyone (although not necessarily understood or interpreted by anyone). In that sense, research tools are at the same time particularly salient technologies, which separates them from other technologies useful to the SSH scholar, and particularly durable methods, which separates them from other parts of the methodological toolbox of the SSH.



Source: Author.

Figure 20.1 *Scientific instruments (research tools) are related to both infrastructures (as technological objects) and research (as methodological objects)*

A tool is a product: it has a name, a stated purpose or function, sometimes a documented and peer-reviewed method, and its developers are accountable for it. It often has a visual identity, a website, documentation, and artifacts dedicated to the community of users. On the infrastructure end of the spectrum, tools cease to be tools when they become systems: machines to administer, services to maintain, etc. Toolmaking becomes maintenance, repair, and support. On the research end of the spectrum, methodological artifacts stop existing as tools when they cease to last because they are not documented, maintained, or even just shared. We find method as pure practice, for instance, as discovery and exploration. Toolmaking then becomes research by traditional means.

The product/system and durable/disposable distinctions are directions more than boundaries. Different people will draw the line at different places. For instance, server technology may look like a tool to a sysadmin while looking like infrastructure to a professor. The key takeaway is that scientific instruments exist because there is an overlap between technology that is durable (support's job) and has a scientific purpose (academics' job). This suggests two ways DCSSH tools come into existence: when a piece of the technological infrastructure becomes visible to researchers by acquiring an identity and stated scientific purpose, or conversely, when the technology supporting a scientific practice becomes durable and is shared and reused by other people. We will provide an overview of different kinds of DC tools for SSH and return to the question of their lifecycle in more detail in Chapter 23.

DC TOOLS FOR SSH ARE NOT MADE FOR THEIR USERS

> [We decided] that we were not going to build a tool based on a needs analysis of other people. We were going to build the tool that we wanted, that we needed to do the experiments that we wanted to do. (Geoffrey Rockwell, in Jacomy, 2023)

There are many reasons why DC tools for SSH are made, and meeting the needs of other people is rarely the main one. That kind of toolmaking is not about capturing a market of users, essentially because the market of penniless students, scholars, journalists, and activists is too small and too risky to justify a full-time job. DC tools for SSH are not made to enrich their creators, but on other premises. Let us look at a few reasons DC tools for SSH are made.

Some tools are brought into existence by their maker to answer their own research questions or meet their own needs. The toolmaker makes the tool for themselves and then shares it with the world as a side effect. There are many reasons for sharing, including that it is sometimes cheap and easy once the tool is already there. But even so, the tool was not made first to be shared; it was shared because it already existed. A good example of this is the convenience tool, developed by a researcher-coder for themselves to address an issue they encounter, and subsequently shared because others have the same issue. Such tools might be as humble as a simple regex wrapped in an HTML form, such as Borra's tools *compare lists*, *URL harvester*, or the *TLD counter*, or they may add a form to a chain of services such as *Expand Tiny URLs* or *GeoIP lookup* for URLs. Convenience tools are not specific to SSH, but the need for reproducibility and the sharing ethos of academia encourages their dissemination in comparison with other contexts like the software industry.

Some tools are made because they implement the methods that the author wants to disseminate. Those may be explicit or implicit vessels for a given method. For example, Borra's

Lippmannian Device is essentially composed of just two form fields that allow users to configure, query, and scrape search engine results. However, by grounding tool practices in Walter Lippmann's (1922) theories on public opinion, the tool allows researchers to identify which web actors align with specific issues and how different issues are discussed by particular actors (Rogers, 2010). Tools may thus disseminate the method of the toolmaker or someone else, and do not have to be *just* a vessel; they may have other goals as well. Those kinds of tools are inherently tied to a specific set of methods because that is their main purpose. For instance, Pierre Ratinaud's text analysis tool *IRaMuTeQ* (Ratinaud, 2009), popular for the analysis of French and Portuguese, is explicitly dedicated to the ALCESTE method (Reinert, 1990), which in this case was conceived by someone else (Ratinaud & Marchand, 2012). Another example is Tiago Peixoto's *Graph-tool* library (Peixoto, 2014), explicitly opinionated, focused on the toolmaker's own algorithms, and reflecting their views about "why you should avoid off-the-shelf methods available in other software packages" (ibid).

Some tools are born as a byproduct of academic practice. A good example from the DCSSH is Bernard Rieder's *Netvizz*, a now-discontinued app that allowed you to download and analyze data from Facebook before the platform closed its API (Rieder, 2013). The tool existed for over ten years and underwent different stages. It started as a simple software experiment; it became an efficient means to understand Facebook's API better; then it became a way for Rieder to nurture his programming skills, and from there it became an annoying responsibility for him because so many people were using it and asking for support. *Netvizz* was not initially made for other people, but it became maintained and redesigned for its users—although if it had ever received funding, the situation may have been very different.

Last but not least, some tools aim at making a theoretical argument. In DCSSH in particular, such tools are entangled with a decades-old literature. As Endres (2017) lists, "scholars have proposed building as theoretical modeling (McCarty, 2004), prototype as theory (Manovich in Ramsay & Rockwell, 2012), prototype as argument (Galey & Ruecker, 2010), API as argument (Sherratt, 2015), design as argument (Galey & Ruecker, 2010; Eyman & Ball, 2015), visualization tools as hermeneutical theory (Rockwell, 2001), digital artifacts as theoretical lenses or 'telescopes for the mind' (McCarty, 2012), building as scholarly intervention (Ramsay & Rockwell, 2012), and tinkering as a process of scholarly reverie, where theory and making collide (Prescott, 2014)". For a recent example, *Gephisto* (Jacomy & Munk, 2024) was created to make the argument that "tools that support critical thinking do not have to be hard to use and hostile to beginners" (ibid, p. 142).

The tools used in the SSH have many other reasons to have been brought into existence than equipping the typical researcher. On the contrary, the toolmaker is generally at the center of the tool's purpose when it is created: because the tool aims at advancing their research, or disseminating their methods, or sustaining their programming skills, or making an argument. If the usefulness to others comes, it generally happens in a subsequent phase (see Chapter 23 about tools' lifecycle). This situation explains why some of the DCSSH researchers' needs are covered by multiple tools while others are unaddressed: meeting those needs is often not a relevant incentive for making a tool. There are two reasons for this. First, from the toolmaker's standpoint, having a tool become popular often comes with more burdens than benefits, and thus is not necessarily something to aim for. Schreibman and Hanlon's (2010) DH toolmaker survey responses "did demonstrate that low adoption rates were something that developers felt hurt the value of their tools, but it was not their biggest concern: level of adoption was ranked fourth among four potential measures of a tool's success". Second, even from the perspective

of meeting many people's needs, there is still a risk that the tool does not meet its public, and that the time and energy invested is lost. In DCSSH at least, toolmakers rarely have the resources to do market research, pivot their product, etc. In the precarious economy of DCSSH toolmaking, a tool is only made when the toolmaker gets a quick return for their time and effort: new data, a paper opportunity, an exciting collaboration, a grant, some publicity … It is only after that stage that the issue of popularity arises.

> In my experience, when I talk to colleagues that have never really used computer assisted text analysis, when they describe what they would like, it is a fantasy. Even if you gave it to them, they would not actually use it. It is what they think they would use. But it is not really what would be useful to them. They do not know enough to be a good, reliable guide. At least we had one grant, a funded project called *Just What Do They Do?* in which we did actually interview a bunch of people. We showed them different screens, and asked what could you do with this and that. … And by [and] large, I have to say, we had more fun, and we got further, just by doing what we wanted; what we thought was interesting. So, in that sense, we have not done the canonical needs analysis, user studies, etc. We were not building things for other people. We were building things for us, and people like us. And that worked! (Geoffrey Rockwell, in Jacomy, 2023)

MANY DC TOOLS FOR SSH ARE TEACHING DEVICES

> Scott Weingart called [*Voyant*] 'the gateway drug for digital humanities.' It is a great entry-level tool. And so that makes it widely used. (Geoffrey Rockwell, in Jacomy, 2023)

Teaching is often what makes a tool popular. It has been the case for *Gephi*, *Netvizz*, *Voyant*, and many others. Some tools were developed first and foremost for teaching, like *Netvizz*, which "initially started in 2008 as a purely technical demonstrator for a course on web programming" (Rieder et al., 2024). Some tools were developed for another purpose but with teaching in mind, like *Gephisto*, mainly conceived as "an intervention into the field of digital methods" (Jacomy & Munk, 2024) but also as a starting point for teaching network analysis. Some were not created with teaching in mind but became used in that context (*Gephi*); and of course, some tools have no particular affinity with the classroom (*DaCy, Minet*).

However, no tool is completely unconcerned with the teaching situation, if only because people need to learn how to use it. As soon as a tool has a user community, teaching material emerges: tutorials, video introductions, FAQs … Some or even all of that material may be from the community itself, not from the toolmakers. Even when the toolmakers do not have teaching in mind, some teachers may use their tool in their teaching, or even as the focus of a lecture or even an entire course. This becomes inevitable as a tool gets popular. For instance, *Gephi* has been the focus of at least two books (Cherven, 2015; Khokhar, 2015), none of which are authored by, or even associated with, its toolmakers. The longer a tool lives, the more probable it becomes that someone teaches it.

Conversely, teaching a tool makes it more popular. The students who have learned the tool will use it, move it to other places, and introduce it to other audiences. This typically takes years, which partly explains why research tools have a lot of inertia: they may be slow to find their audience, but they may retain an audience for a very long time. Even years after the tool is abandoned, someone may need to use it, for instance, to replicate an academic publication in which the tool was used as part of the methodology. When teachers use tools as part of their

curriculum, developers may receive urgent bug fixes from students or faculty who discover issues, even after the developer has completed or left the project without being informed of its ongoing or classroom use. Teaching and popularity are in a reinforcement loop: teaching the tool makes it more popular, and popularity increases the chances that a tool gets used.

When a tool becomes popular because it is taught, its popularity will follow the localization of its teaching: a given university, academic field, language … A tool's user community is not necessarily unified. We should rather use the plural. User communities are not neutral or universal, but situated. Each one may cultivate its partial perspective on the tool: its purpose, strengths and weaknesses, preferred settings, and good practices. For example, *Gephi* is considered very user-friendly by network scientists but difficult to learn by digital humanists. The former may use it for its visuals but find the embedded algorithms too basic, while the latter may use it as their primary platform for computing network statistics. The former may understand networks as statistical models (Peixoto, 2023), while the latter may see them as a means to describe phenomena (Venturini et al., 2021). Some user communities form their own "epistemic culture" or subculture (Knorr Cetina, 1999; see also Jacomy & Jokubauskaitė, 2022).

Despite the impact of teaching on tool popularity, from the standpoint of the teacher, there are drawbacks to introducing a single instrument, because it is not guaranteed that the students will be able to use it in their jobs, typically because the organization is committed to an alternative solution. As a result, the teacher may choose to cover multiple tools for a single purpose. This has the benefit of making the specific choices visible by comparison. This kind of concern constitutes the specificity of the teaching situation. Some tools are better attuned to it, for instance, because they are useful as ways to introduce other tools, even though those other tools may be better adapted to other contexts.

In the case of DCSSH in particular, quite a lot of tools were designed by someone who, paradoxically, was not comfortable with existing technology. Some tools are made by scholars for themselves. *Netvizz* was initially developed by Rieder (2013) as a software experiment, a way to understand technology and improve coding skills; *Gephi* was initially prototyped by Jacomy for himself and DCSSH researchers, and only subsequently passed on to the professionally trained developers Mathieu Bastian and Eduardo Ramos Ibañez. *DMI-TCAT* was made in response to the data dashboards of marketing companies, with e.g. vanity metrics rather than social science metrics built-in (Borra and Rieder, 2014). As we already mentioned, discomfort with technology can motivate someone to make a tool because they aim to benefit from the *process* of toolmaking rather than from its result. In addition, the toolmaker might also be motivated by the fact that existing tools are not suitable for the DCSSH: too complicated, too expensive, sacrificing too much quality for performance … Some DCSSH toolmakers are more similar to a student or researcher than a professional developer employed at a tech company. For that very reason, the tools designed by that kind of toolmaker are naturally attuned to the classroom. They speak to beginners because they open the way, because the toolmaker acted as a passeur of methods and algorithms, or simply because the toolmaker is the one who knows how to program.

The toolmakers themselves may be teaching their tool, whether to students or researchers—possibly as a way to promote it, but as we will see in the next chapter, the desire to popularize one's tool should not be taken for granted. More pragmatically, the toolmaker may be looking for feedback. Through this, one can learn in which situations the tool is efficient or inefficient, when it breaks, and which features are understood or misunderstood by the users.

Quite often, the teaching situation may also inspire new features when users repurpose the tool in unexpected ways. To some scholars, toolmaking is teaching by other means.

> Any time somebody asks me: Would you Zoom in to my class and give an intro to *Voyant*? I say Yes. I can give those in my sleep, now. I give them on a regular basis. I just come in and talk about *Voyant*, or I run a one-hour session. I get a certain amount of feedback from that. (Geoffrey Rockwell, in Jacomy, 2023)

TOOLS EMPOWER USERS … AND MAKERS

> [*Voyant*] empowers people. It allows them to get a taste of text analysis and visualization, including things like topic modeling and principal component analysis. It allows them to get a taste, but there is something else. We have a bunch of wacky tools, and the wacky tools send an interesting message: you can play. It empowers the people who have to teach in the digital humanities. (Geoffrey Rockwell, in Jacomy, 2023)

Scientific instruments' empowerment does not necessarily support "good" science, as critics have pointed out (Drucker, 2014; Masson, 2017; Tenen, 2016); this is regardless of what "good" may refer to, as there is simply no way to contain the use of a tool within any predetermined set of practices. When you grab a tool, you contribute to shaping its function and may be held accountable for it. Not even the toolmaker has complete control over what the tool allows and disallows. A developer may nudge its users, but often cannot dictate imagined or actual uses. What the empowerment consists of is partially discovered through actual usage by different people and is never fully known in advance. The direction taken by the tool-user assemblage, i.e. the equipped researcher, emerges from a material dialogue between the researcher's partial perspective and the theories infrastructured into the instrument. The opportunity to repurpose a tool is in the very nature of the empowerment it brings.

Maybe less obviously, the tool also empowers the toolmaker. Not only because a successful tool may give power to its toolmaker(s) but also, and more importantly, because the *process* of making the tool is scientifically fruitful. Moreover, these empowerments are not distinct ones but entangled, because in many ways the tool offers the end user the opportunity to re-trace the traces left by the toolmaker during the making of the tool. Rieder et al. (2024) make this precise point:

> On the first level, … research software enters into a conversation with existing knowledge and methodology … [and] is heavily dependent on the labor and dedication that goes into creating the thousands of building blocks that inform development every step of the way. … [On a] second level, … the process of diving into specifications, of bumping into technical restrictions, and of discovering 'tricks' and loopholes [is] epistemically productive, in the sense that it may lead to new pathways that turn into specific features or methods further down the line. … On a third level, end users come into play. … User and author both contribute to the evolution of a research tool; but one could also say that the tool *develops the user*, by presenting and framing particular approaches to research they may not yet be familiar with. … Taken together, these three entanglements create a space of action for toolmakers that is bounded and 'populated' by other actors.

Here is an example from the making of *Gephi* (Bastian et al., 2009). The tool proposes a number of "layout algorithms" to the user, aimed at arranging the network's nodes in the 2D plane

of the screen based on how they are connected (Venturini et al., 2021). The algorithms available at the beginning of *Gephi*'s life required know-how about which algorithm to choose in a given situation, and how to set its parameters. This, among other reasons, prompted the toolmakers to develop their own algorithm, *ForceAtlas2* (Jacomy et al., 2014). In the experimental process of finding a good recipe, they rediscovered an efficient way to balance that kind of algorithm, previously observed by Andreas Noack (2007) under the name of "edge-repulsion". In short, it entails a special way of accounting for edges in one of the algorithm's equations that seems to produce a better visual separation of clusters. This led them to rethink whether the aesthetic qualities of popular algorithms such as *ForceAtlas2* and the *LinLog* (Noack, 2007) are actually due to the "energy model" (ibid), i.e. the general profile of the algorithm's equation, as generally accepted, or to edge-repulsion itself (Jacomy, 2021; Venturini et al., 2021). Making *Gephi* was, for its toolmakers, an entry point into an academic discussion about graph drawing and clustering.

The empowerment of the toolmaker is an important externality of toolmaking, both to the academic community and to the toolmaker themselves. To the academic community, it may benefit from the emergence of (infrastructured) theories that may not come to light by other means. It enhances the epistemic diversity of a field. It is also important to the toolmaker themselves because the scientific knowledge they gain from it (among other benefits) contributes to solidifying their identity as an actor in academic research, which is necessary for maintaining their status as a scientific toolmaker.

> [*Voyant*] also empowered Stéfan and I to build a career around this, in a way that would have been very difficult to do if we had not. You look back on your life and you go: it could have gone in very different directions. (Geoffrey Rockwell, in Jacomy, 2023)

TOOLS BEAR THEORY

> The thesis of our book, *Hermeneutica*, is that, in some sense, tools bear theory in a different way than texts. A hermeneutical text says: here is how you interpret a text, here is how you do interpretation. A tool bears a theory but bears it differently. We were playing with this idea of 'tools can be theories,' and we turned around and started telling people that. Our papers were telling people that tools are theories. (Geoffrey Rockwell, in Jacomy, 2023)

Toolmaking is part of the "social life of methods" (Savage, 2013), embedded in social practices and relationships, influenced by power dynamics and cultural values, but also influencing them; and not just when it comes to the big notorious tools. "Every CSV is an expression of power, every API is an argument" (Sherratt in Endres, 2017). (Research) software does not just provide technical functionality but encodes methodological and epistemological considerations, just as the data used in computational systems have epistemological repercussions for research (Borra, 2023). It may, for instance, influence how variables are selected and operationalized, which analyses can be conducted, and how the results are interpreted and presented (Uprichard et al., 2008). A perfectly atheoretical tool would be useless and powerless, because it could not make a difference to anything, as that would be committing to one method over other options. From this stems the argument that tools bear theory. This isn't making a claim about what theory should be. Rather, it is stating a basic fact we must accept,

or else we could not properly analyze how scientific instruments affect knowledge production. Tools have epistemic orientations that affect how research is conducted and what kind of knowledge is produced (Borra & Rieder, 2014). When we use research tools, we are implicitly accepting certain theoretical views about how the world works.

One could simply say that tools have *scientific agency*, the ability to shape research processes and outcomes; or more precisely, that the user-tool assemblage can displace the goals of both the user and the tool (Latour, 1994). Yet framing tool influence merely as "agency" risks reducing it to a simple impact on research. This perspective fails to capture the crucial point that, for toolmakers, theory is not simply *applied* through tool creation, but rather *constituted* within the very process of toolmaking. The theoretical content, therefore, is inextricably linked to the practical act of design and development. In that sense, "theory" becomes less a pre-existing framework and more an emergent property of the toolmaking practice itself.

We are used to encountering theory as text, as a structured way of explaining phenomena, involving well-defined concepts and their relations, like causal statements specifying mechanisms about how and why certain things occur, perhaps with interpretive elements. But within the tool, theory has a different mode of existence from text; let us call it *infrastructured* to highlight that there is a material difference (following similar ideas in Gray et al., 2018). The materiality of the tool comes with losses and gains over the materiality of text. It loses the discursive properties of the text: the infrastructured theories (theories embedded within technological systems) are less explicit, less discussable. The theoretical load of a tool requires specific work to be criticized, or a different, non-textual approach to criticism; or it could go undetected and escape criticism entirely. In exchange for this loss, infrastructured theories gain *affordances* (Gibson, 1966), i.e. material ways to guide the user without requiring any *a priori* explanation or justification. In short, infrastructured theories are not read but enacted and experienced. Many may be better convinced by experiencing than by reading, and at the same time, the subjective element of experience may make it more difficult to share that knowledge, in addition to the necessity of putting the theory into words. Using a tool can convince you of things you may not be able to formulate; it can convey patterns that you are not fully conscious of having internalized, nonverbal information we typically label as know-how, as expertise. This information constitutes knowledge nonetheless and is theoretical insofar as it gives currency to some explanations to the detriment of others. "In the context of history or literary study, 'theory' doesn't predict, but it does explain. … To say that software is a theory is to say that digital works convey knowledge the way a theory does, in this more general sense" (Ramsay & Rockwell, 2012).

The toolmaker is well aware that the affordances of their tool make some research agendas easier and others more difficult, notably when they shape the tool for a given situation (their own usage, a data sprint, a project …). The toolmaker is inevitably drawn to a somewhat normative position, whether they like it or not. They will typically shape the affordances to improve the tool, for instance to enforce better research practices (Peeters & Hagen, 2022), but what they deem good might not work for someone else, which is why one should always question how the tool addresses one's needs. Even seemingly minor design decisions—such as database structures, default parameters, or the order of options presented—can have significant effects on research (Borra & Rieder, 2014). Such choices influence how researchers interact with data and shape the conclusions they draw (Borra, 2023). As the tool will inevitably be normative to some extent and as the toolmaker is generally aware of it, they will rarely paint their tool as agnostic. They may even unapologetically assert the methodological

direction they deem best through "prescriptive gestures" (Rieder et al., 2024) and mitigate the related accountability by informing the user about it. Alternatively, some tools resist making choices by providing as many options as possible, for instance *SciKit-Learn* (Pedregosa et al., 2011). As Rieder et al. (2024) write, "while terms like 'bias' still suggest that epistemic neutrality is a possibility or at least a desideratum, we suggest that it is both inevitable and productive that commitments, idiosyncrasies, and contextual embeddings shape tool-making and, consequently, tools themselves. Inevitable because any design activity involves a plethora of choices, and productive because tool-making is a site of expression and negotiation that can stimulate collective processes of methodological debate and sense-making".

The theory-bearing ability of scientific instruments is often acknowledged when it comes to criticizing them. That literature within the DCSSH recurrently paints tools as Trojan horses, generally of positivism. For instance, Eef Masson (2017) writes: "with the introduction of digital research tools, and tools for data research specifically, humanistic scholarship seems to get increasingly indebted to positivist traditions … those tools, more often than not, are borrowed from disciplines centered on the analysis of empirical, usually quantitative data. Inevitably, then, they incorporate the epistemic traditions they derive from". Or Johanna Drucker (2014): "Most, if not all, of the visualizations adopted by humanists, such as GIS mapping, graphs, and charts, were developed in other disciplines. These graphical tools are a kind of intellectual Trojan horse, a vehicle through which assumptions about what constitutes information swarm with potent force". We take issue with that perspective. Although the Trojan horse metaphor fairly accounts for a tool's ability to obfuscate its theoretical load, we contest that this load has been intentionally hidden and conveyed by the toolmaker for the purpose of influencing and deceiving. We make this argument in different ways through this piece, but in a nutshell, although the toolmaker's perspective is necessarily partial, they are not in full control of what theory ends up embedded in their tool. When they want to operationalize a theory, it does not work; and conversely, they end up implementing theories they are not even aware of. The latter was the case with the earlier mentioned *Lippmannian Device* that originated from the particular use of a Google scraper, which, reconfigured and loaded with theory, became a means to show actor partisanship and issue commitment (Rogers, 2010). Rieder et al. (2024) thus argue for a "tool criticism" to not just uncover hidden epistemological assumptions, but also to make informed design choices and enhance transparency in research tools. Tool criticism is not merely reactive; it actively shapes how tools are designed, documented, and communicated.

It is thus necessary but not sufficient to acknowledge that tools have a theoretical load or "epistemological commitments" (Rieder & Röhle, 2017). The infrastructured theory remains "uncanny", as Marres and Gerlitz (2016) put it: "A key characteristic of the methodological uncanny is that it is not necessarily clear, which analytic purposes digital tools may serve, what research objectives they may align with or what disciplinary agendas they enact" (ibid, p. 26). What the infrastructured theory consists of should never be taken for granted. Even when the toolmaker has documented the theories and methods at stake in their tool (a great move, by the way), one should not take it at face value, essentially because the tool's alignment may remain unseen by the makers themselves. This is a concern when it comes to criticism, but also an opportunity to repurpose technology in ways that benefit the researcher, which has been instrumental to some disciplines like (digital) media studies. Indeed, "much of the debate about digital methods in social media studies has focused on the possibility of the repurposing of digital devices (Rogers, 2009). Sociologists have drawn attention to the instability and under-determinacy of digital research methods themselves, proposing notions such as

plastic methods (Lury, 2012) and live methods (Back and Puwar, 2012)" (Marres & Gerlitz, 2016). Infrastructured theories do not boil down to hidden influence, as their epistemic polyvalence also offers you, the scholarly user, opportunities to bend and reshape them.

From the maker's standpoint, the practice of toolmaking aims to operationalize theories while at the same time acknowledging the user's freedom to use the tool in ways that challenge the theories supposedly baked into it. If a tool bears theory, then we must accept that this theory might well be discovered or reinvented (depending on how you see it) by other researchers. Instruments shape science, albeit in ways that one cannot take for granted (toolmakers included), which should always be accounted for. Paraphrasing one of us (Borra, 2023):

> [Toolmaking may] be viewed as 'a contribution to academic knowledge production' (Rieder et al., 2024, para. 54). Programming research instruments is more than just a way to scope and reuse data flows and activities in sociotechnical systems; it also attempts to operationalize a given research technique and establish a case for it. The combination of theoretically informed concerns and toolbuilding constitutes an academic genre that connects critical studies with technical practice (Agre, 1997) and is a vital part of knowledge production, even though it may not always be recognized as such (Jay et al., 2021; Katz & Stall, 2021). Building tools allows one to engage with the way data is being generated, 'infrastructured,' and used in practice (Gray et al., 2018)

The theories structured into a tool during its making take a pragmatic turn by nature. As William James wrote (1907), "theories thus become instruments, not answers to enigmas, in which we can rest". Structured theory then refers to the methodological and epistemological commitments embedded in tools, shaping research through their affordances rather than through explicit argumentation. They work as means to make sense of the world, not as definitive explanations but as vectors of momentary epiphanies, precisely because they do not consist of stable textual statements, and therefore cannot become resting places for our thinking.

REFERENCES

Agre, P. E. (1997). Toward a critical technical practice: Lessons learned in trying to reform AI. In G. Bowker, S. L. Star, L. Gasser, & W. Turner (Eds.), *Social Science, Technical Systems, and Cooperative Work: Beyond the Great Divide* (pp. 131–157). Psychology Press.

Back, L., & Puwar, N. (2012). A manifesto for live methods: Provocations and capacities. *The Sociological Review*, *60*(1_suppl), 6–17.

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. *Proceedings of the International Aaai Conference on Web and Social Media*, *3*(1), 361–362.

Borra, E. (2023). *Object-activity: Repurposing the dual nature of web data for digital research.* [PhD Thesis].

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. *Aslib Journal of Information Management, 66*(3), 262–278.

Cherven, K. (2015). *Mastering Gephi Network Visualization.* Packt Publishing Ltd.

Drucker, J. (2014). *Graphesis: Visual Forms of Knowledge Production*. Harvard University Press.

Endres, B. (2017). A literacy of building: Making in the digital humanities. In J. Sayers (Ed.), *Making Things and Drawing Boundaries: Experiments in the Digital Humanities* (pp. 44–54). University of Minnesota Press.

Eyman, D., & Ball, C. (2015). Digital humanities scholarship and electronic publication. In J. Ridolfo & W. Hart-Davidson (Eds.), *Rhetoric and the Digital Humanities* (pp. 65–79). University of Chicago Press.

Galey, A., & Ruecker, S. (2010). How a prototype argues. *Literary and Linguistic Computing*, *25*(4), 405–424. https://doi.org/10.1093/llc/fqq021

Gibson, J. J. (1966). *The Senses Considered as Perceptual Systems.* Allen and Unwin.

Gray, J., Gerlitz, C., & Bounegru, L. (2018). Data infrastructure literacy. *Big Data & Society, 5*(2). https://doi.org/10.1177/2053951718786316

Jacomy, M. (2021). *Situating visual network analysis.* [PhD thesis].

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. *Reticular.* https://reticular.hypotheses.org/8039

Jacomy, M., & Jokubauskaitė, E. (2022). Unblackboxing Gephi: How a user culture shapes its scientific instrument. Working paper. https://hal.science/hal-03729114/file/Unblackboxing%20Gephi%20Rev%201.pdf (accessed 26/08/2024).

Jacomy, M., & Munk, A. K. (2024). Interfering with the black-box-tradeoff model: Gephisto, a one-click Gephi for critical technical practice. *Convergence*, *30*(1), 142–166. https://doi.org/10.1177/13548565221129053

Jacomy, M., Venturini, T., Heymann, S., & Bastian, M. (2014). ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. *PloS One, 9*(6), e98679.

James, W. (1907). Lecture II: What pragmatism means. In W. James (Ed.), *Pragmatism: A new name for some old ways of thinking* (pp. 43–81). Longmans, Green and Co. https://doi.org/10.1037/10851-002

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. *International Journal of Digital Curation*, *16*(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Katz, D. S., & Stall, S. (2021). Software citation. *Information Services & Use, 41*(1–2), 39–42. https://doi.org/10.3233/ISU-210108

Khokhar, D. (2015). *Gephi Cookbook.* Packt Publishing.

Knorr Cetina, K. (1999). *Epistemic Cultures: How the Sciences Make Knowledge.* Harvard University Press.

Latour, B. (1994). On technical mediation. *Common Knowledge, 3*(2), 29–64.

Lippmann, W. (1922). *Public Opinion*. MacMillan. https://doi.org/10.1037/14847-000

Lury, C. (2012). Going live: Towards an amphibious sociology. *The Sociological Review, 60*(1_suppl), 184–197.

Marres, N., & Gerlitz, C. (2016). Interface methods: Renegotiating relations between digital social research, STS and sociology. *The Sociological Review*, *64*(1), 21–46.

Maslow, A. H. (1966). *The Psychology of Science.* Harper & Row.

Masson, E. (2017). Humanistic data research: An encounter between epistemic traditions. In M. T. Schäfer and K. van Es (Eds.), *The Datafied Society: Studying Culture through Data* (pp. 25–38). Amsterdam University Press.

McCarty, W. (2004). Modeling: A study in words and meanings. In S. Schreibman, R. Siemens, & J. Unsworth (Eds.), *A Companion to Digital Humanities* (pp. 254–270). Blackwell Publishing.

McCarty, W. (2012). A telescope for the mind? In M. K. Gold (Ed.), *Debates in the Digital Humanities* (pp. 113–123). University of Minnesota Press.

Noack, A. (2007). Energy models for graph clustering. *Journal of Graph Algorithms and Applications*, *11*(2), 453–480.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. *Computational Communication Research*, *4*(2), 571–589.

Peixoto, T. (2014). Graph-tool: Efficient network analysis with Python. https://graph-tool.skewed.de (accessed 14/08/2024).

Peixoto, T. P. (2023). *Descriptive vs. Inferential Community Detection in Networks: Pitfalls, Myths and Half-truths.* Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Prescott, A. (2014). Made in Sheffield: Industrial perspectives on the digital humanities. *Studies in the Digital Humanities*, (1).

Ramsay, S., & Rockwell, G. (2012). Developing things: Notes toward an epistemology of building in the digital humanities. In M. K. Gold (Ed.), *Debates in the Digital Humanities* (pp. 75–84). University of Minnesota Press.

Ratinaud, P. (2009). IRaMuTeQ: Interface de R pour les Analyses Multidimensionnelles de TExtes et de Questionnaires. http://www.iramuteq.org (accessed 14/08/2024).

Ratinaud, P., & Marchand, P. (2012). Application de la méthode ALCESTE à de "gros" corpus et stabilité des "mondes lexicaux": analyse du "CableGate" avec IRaMuTeQ. *Actes des 11eme Journées internationales d'Analyse statistique des Données Textuelles, 3*, 835–844.

Reinert, M. (1990). Alceste une méthodologie d'analyse des données textuelles et une application: Aurelia De Gerard De Nerval. *Bulletin of Sociological Methodology/Bulletin de méthodologie sociologique, 26*(1), 24–54.

Rieder, B. (2013). Studying Facebook via data extraction: The Netvizz application. In *Proceedings of the 5th annual ACM Web Science Conference* (pp. 346–355). Association for Computing Machinery.

Rieder, B., & Röhle, T. (2017). Digital methods: From challenges to bildung. In M. T. Schäfer & K. van Es (Eds.), *The Datafied Society: Studying Culture Through Data* (pp. 109–124). Amsterdam University Press.

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. *Convergence*, *30*(1), 216–235. https://doi.org/10.1177/13548565221127094

Rockwell, G. (2001). The visual concordance: The design of Eye-ConTact. *Text Technology*, *10*(1), 73–86.

Rockwell, G., & Sinclair, S. (2016). A model theory: Thinking through hermeneutical things. In G. Rockwell & S. Sinclair (Eds.), *Hermeneutica: Computer-Assisted Interpretation in the Humanities* (pp. 149–166). MIT Press.

Rogers, R. (2009). *The End of the Virtual: Digital Methods* (Vol. 339). Amsterdam University Press.

Rogers, R. (2010). Internet research: The question of method—a keynote address from the YouTube and the 2008 election cycle in the United States Conference. *Journal of Information Technology & Politics*, *7*(2–3), 241–260. https://doi.org/10.1080/19331681003753438

Savage, M. (2013). The 'Social life of methods': A critical introduction. *Theory, Culture & Society*, *30*(4), 3–21.

Schreibman, S., & Hanlon, A. M. (2010). Determining value for digital humanities tools: Report on a survey of tool developers. *Digital Humanities Quarterly*, *4*(2). https://www.proquest.com/docview/2555208512/abstract/8B9E2E7ABEE94C7EPQ/1 (accessed 01/09/2024).

Sherratt, T. (2015). Unremembering the forgotten. Keynote Address, *International Conference of the Alliance of Digital Humanities Organisations*.

Tenen, D. (2016). Blunt instrumentalism: On tools and methods. *Debates in the Digital Humanities*, *216,* 83–91.

Uprichard, E., Burrows, R., & Byrne, D. (2008). SPSS as an 'Inscription Device': From causality to description? *The Sociological Review, 56*(4), 606–622. https://doi.org/10.1111/j.1467-954X.2008.00807.x

Venturini, T., Jacomy, M., & Jensen, P. (2021). What do we see when we look at networks: Visual network analysis, relational ambiguity, and force-directed layouts. *Big Data & Society, 8*(1).

21. Meet the DCSSH toolmakers

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

The makers of DC tools for SSH often operate in challenging institutional contexts, straddling the divide between academic research roles and technical support—which is rarely part of their job description. Knowing about this situation is valuable to scholars because it helps them better appreciate why certain tools exist while others do not, why some tools receive consistent maintenance while others languish, and how to more effectively engage with toolmakers when seeking support or requesting features.

This chapter looks into the institutional organization of universities, which typically enforces a strict separation between support staff and academic researchers, creating tensions that affect tool development and maintenance. This explains why finding and maintaining suitable research tools can be challenging in the DCSSH, where specialized tools cannot always be borrowed from other disciplines or from the industry.

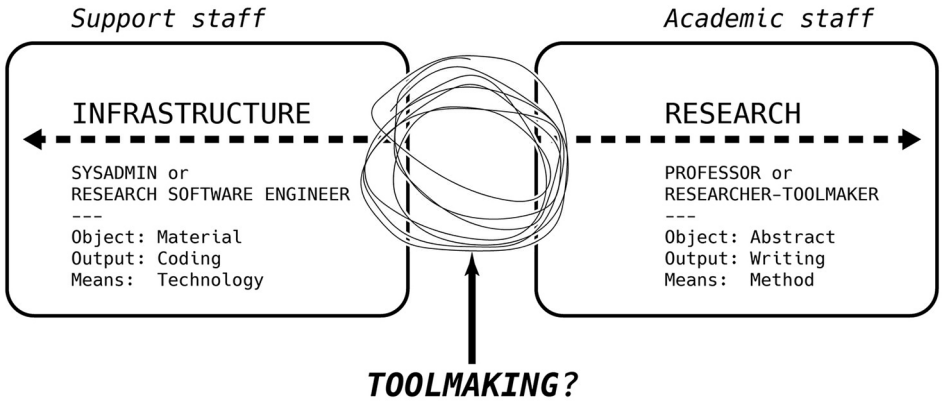
Understanding the professional contexts and challenges of DCSSH toolmakers allows scholars to better navigate the tool landscape: knowing how to effectively communicate their needs through appropriate channels, recognizing the value of building relationships with the toolmaking community, and appreciating the complex trade-offs that shape tool development.

WHO MAKES TOOLS FOR DCSSH?

> My name is Geoffrey Rockwell. I am professor of Philosophy and Digital Humanities at the University of Alberta. … One of the things I love to do is to teach humanities students to program. … I wish I had the time to be a better programmer. (Geoffrey Rockwell, in Jacomy, 2023)

The instruments SSH scholars buy from the market are built and maintained by hired professionals; however, free research tools come from a different place and have a different existence. One cannot simply assume that DCSSH toolmakers make tools for a living. Looking at who they are, you find research engineers who maintain infrastructures for a living—but it does not necessarily entail tool design; academics who make tools as a byproduct of their research—but making and maintaining tools is outside their job's mission statement; and many in-between situations where the toolmaker is a scientist-coder (Perkel, 2021) with an unusual position and/or mission statement, or an unofficial deal with their management about their activity.

DCSSH toolmaking entails some amount of identity crisis because it is at odds with the institutional organization of science, particularly at the university level (less so at the level of a particular research group, as we will see). Indeed, universities generally enforce a strict separation between two kinds of employees: support (IT services, accountants, research engineers…) and academic staff (researchers, professors…) (see Figure 21.1). Let us acknowledge that there



Source: Author.

Figure 21.1 *The infrastructure-research dichotomy as seen by scientific institutions: as a divide supporting a division of labor between, for example, the research software engineer and the professor*

is a strong case in favor of this distinction, notably as a means to protect the researchers' ability to self-determine their scientific agenda while simultaneously keeping necessary technical skills within the university. The mission statements, key performance indicators, and salaries are sensibly different on each side of this border. The difference between a research software engineer (RSE) and a professor, for example, both of whom could be making tools, assumes a division of labor between material/abstract, coding/writing, technology/methodology, etc. Unfortunately, these dichotomies conflict with the practice of toolmaking because, in a nutshell, the central tenet of scientific instrument design is that technology and methodology are two sides of the same coin.

Going against this perspective, the practices of those who make tools for DCSSH are distributed on a spectrum from infrastructure to research. Contrary to what the division of labor in universities suggests, there exists a continuum of toolmaking profiles between the support and academic positions (Figure 20.1 in Chapter 20). On the infrastructure end, toolmakers think of themselves as civil servants and maintain the technical apparatus necessary to do research, but do not wish to write academic publications (even though they often end up doing so to some extent). Baxter et al. (2012) coined the term "Research Software Engineer" to identify such a profile, someone who "relishes [the] challenge of not just developing code to solve a problem but doing it well" and "want[s] to be recognized for producing tools which others rely on for research" (ibid, p. 2). On the research end, we find academics who see toolmaking either as an autoethnographic and critical practice necessary to their research, as a way to produce experimental demonstrations, and/or as a form of knowledge production by itself. In short, it is an integral part of their own research (while the RSE supports the research of others). They find ways to valorize their tools academically, either through proxy papers (meant to be cited in place of a given tool), books, praxes (reflective activities integrating theory and practice), or by making the tool itself citable, for instance with a DOI (although the lack of DOI should not

prevent citing; see Boettiger, 2013). Baxter et al. call this profile the "Researcher-Developer", someone who "wants to be judged on their scientific output, is a researcher at heart, however, develops a lot of code due to the nature of their research" (ibid). Outside of this spectrum, toolmakers may be from the private sector or civil society and aim to equip other publics (citizens; experts; newsrooms; activists …) and incidentally, DCSSH scholars happen to reuse and repurpose their tools.

ACADEMIC TOOLMAKING IS NOT (YET) A CAREER PATH

Academic toolmaking is lacking a clear career path because it conflicts with the support/science divide of academic institutions (Figure 21.1). Baxter et al. (2012) highlight this issue as one of their main reasons for proposing the RSE figure. They invented that name in their paper to draw attention to the hidden labor of software engineers in academia, to better support the maintainability of scientific computing, and to help address other similar issues. As they explain, "this absence of a career path is due to the lack of recognition and appreciation of the output of software development work in the academic system of assessment, which measures success only through published peer-reviewed publications. The Science Code Manifesto [(Barnes et al., 2013)], whose founding signatories include Peter Norvig [prominent computer scientist and then Director of Research at Google], Joseph Jackson [representing the Open Science Alliance], and Victoria Stodden [statistician notably known for her work on science reproducibility], talks about this explicitly: 'Software is an essential research product, and the effort to produce, maintain, adapt, and curate code must be recognized. Software stands among other vital scientific contributions besides published papers'". The discussions about the issues of producing and maintaining code in science are not new, and the lack of a well-defined career path is often considered the central problem.

The lack of a career path for toolmakers jeopardizes the quality of the tools available to the DCSSH, particularly tools dedicated to the specific needs of the SSH, which cannot be borrowed from any other field or from the industry. The tools born within DCSSH, which may be the most valuable to the field, are created and maintained by support or academic employees who have an uncertain and uneasy career path as toolmakers. The main threat to their toolmaking activity looks different on either side of the divide, but it similarly consists of pressure to quit doing the things that belong to the other side. The RSE will typically be told to stop maintaining their own tool and instead support accredited researchers; they will be denied their own agenda. Mirroring this situation, the researcher will be pressured to quit coding or documenting their tool and instead teach and publish peer-reviewed articles; their technological production will be seen as worthless. The relative precariousness of DC tools for SSH largely roots in this contradiction between the practice of toolmaking and the professional paradigm of universities, which has at least the benefit of suggesting a way forward to improve the quality of our scientific instruments: creating an identified career path for DCSSH toolmakers.

Since Baxter et al.'s publication in 2012, RSE positions have been increasingly normalized. Nevertheless, the support/academic staff divide is so deeply baked into Western universities (at least) that this normalization has displaced the problem rather than solved it. In our experience, from the standpoint of a researcher in the European Union in 2024, collaborating with RSEs remains complicated because they are typically paid by structural funding, where a lot

of bureaucracy gets involved to settle who can benefit from their help, for what purpose, and for how long. Counterproductively, this may in turn lead to decoupling software development from actual research.

Ultimately, the lack of a dedicated career space for the toolmaker is at the same time a symptom of the multifaceted issues of toolmaking in academia and a source of problems on its own. It derives from the airtight silos of support versus academic positions; from the invisibility of code maintenance (we develop this point in Chapter 23); and from technology not being valued as academic production in SSH (see Chapter 24).

TENSIONS INHERENT TO DCSSH TOOLMAKING

> The very first tool that I made with John Bradley, TACTWeb, was the beginning of the [toolmaking] path. … We presented it at ALLC/ACH Paris in 1994. And for what it is worth, I can still remember a very important person in the community basically saying that this is not digital humanities, that there is not an argument here, that this [is] just a pretty visualization, and that it is cute but useless. … There was a little bit of a surprise for me to suddenly realize that my own community was going like: 'Eh! This is not serious.' I mean, to be fair, this was just one person. (Geoffrey Rockwell, in Jacomy, 2023)

In academia, the toolmaker faces two seemingly antagonistic forms of gatekeeping. The first challenges the scholarship or scientific nature of software production, while the second posits that the tool has undesirable methodological principles.

The latter kind of gatekeeping comes in two flavors. In some cases, the critique simply opposes the method baked into the tool, for instance by framing an algorithm implemented in Gephi as "harmful" (Peixoto, 2023), while in other cases, it blames ease of use for undermining good methodological practices. For instance, Tenen (2016) argues that "any attempt to further repackage or to simplify the tool can only add another layer of obfuscation. It follows, then, that without a critical discussion about implicit methods, out-of-the-box tool use is best treated with a measure of suspicion". This criticism paints the toolmaker as a foolmaker, someone who, through their tools, makes the "lazy" able to do things they "should not", at the expense of the general interest of the academic community.

These criticisms are legitimate and healthy. The sole virtue of being used by scholars does not inherently make a tool academically fruitful and methodologically sound. The toolmaker is generally aware of this and tries to ground their software production, typically by publishing it according to FAIR principles so that its methodologies can be inspected (Lamprecht et al., 2020), by referencing relevant peer-reviewed publications in the documentation and in the user interface, and so forth. Their efforts may be deemed sufficient or not; we can safely assume that most toolmakers consciously face the accountability inherent to tool authorship. However, this accountability interferes with a tension that lies somewhere else entirely.

The tool inevitably escapes its maker to some extent. It allows unintended things; it gets repurposed by users in unexpected ways; it has a life of its own. The maker is only partially responsible for what people do with their tool—is it even "their" tool once reappropriated by others? The more powerful the technology, the greater the potential for misuse. Furthermore, the spectrum of digital literacy being particularly broad, a given software tool is always too easy for someone and too difficult for someone else. And by the nature of the academic community, the method it embeds is always the wrong one in someone's eyes. Even though

toolmakers try to nudge users in the "right" direction of open science or privacy by design (Peeters & Hagen, 2022), there is no guarantee that most users will follow the design intents. Because it partly lies outside of the toolmaker's control, their accountability is in tension with the user's freedom.

Most toolmakers generally try to address criticism, yet they may disregard it when it fails to account for the relative independence of the tool from its maker. This independence is intimately known to some toolmakers, yet difficult to account for. Part of it is due to the constant necessity to compromise in the time-constrained, resource-deprived environment of a DCSSH research team, which is easy to understand. But more importantly, and less easily construed, the data and the algorithms themselves resist the toolmaker. Even in the best situation, the toolmaker has no other choice but to compromise with the materiality of the computer: this algorithm is too slow or too memory-intensive; that data structure does not support all the necessary cases; those seemingly complementary algorithmic techniques turn out to be irreconcilable; that visualization technique creates anti-patterns, confusing half of its users; or more prosaically, yet another social media platform closes its API. The toolmaker often does what they can rather than what they wish, and part of the criticism they receive is actually aimed at things outside of their control or that they cannot afford timewise or resource-wise.

Who takes responsibility for the way the tool gets used? The toolmaker can also be exposed to risks related to unforeseen or unwanted applications of their tool. When the tool allows the user to do something illegal or cause harm, will the toolmaker be held accountable? For instance, the social media company Meta (Facebook, Instagram) has been filing lawsuits against web scraping (Hutchinson, 2024), just as X has (ACLU, 2024). Many DCSSH researchers who relied on web scrapers for their research may worry about it, and their research may already be impacted by those lawsuits, regardless of their outcome. Who faces the risk, the tool or the scholar? What kind of technology is the DC tool for SSH? Is it more like a knife, mainly a commodity albeit a weapon in power, where agency lies in the hand holding it? Or is it like a highly toxic chemical, mainly a threat albeit useful in rare situations, harmful by the sole virtue of its existence? Are some DC tools for SSH so dangerous that they should be forbidden, for legal, methodological, or ethical reasons? "What a plague you unleashed upon us!" Johanna Drucker (a prominent DH scholar) once told one of us (Jacomy) for co-creating the tool Gephi. Accountability gets distributed among the user, the tool itself, and its maker(s), and that distribution can never be taken for granted (Latour, 1994). When using a tool in research involves a risk, multiple actors may be exposed: from the researcher to the toolmaker, from the funder to the university. Will the toolmaker be backed by their institution if someone uses their tool to breach a platform's terms of service—notably because one may usually not make derivative works—or even countries' policies or laws? This is, for example, the case of DMI's Censorship Explorer. Because of that kind of risk, the tool puts its maker in tension with other actors. Toolmakers may respond to it in very different ways, from ignoring it entirely, to limiting impact by trying to keep their tool relatively confidential, to mitigating the risk itself either by following guidelines such as the FAIR principles (Lamprecht et al., 2020) or by simply abstaining from implementing certain features.

TOOLMAKERS GET SUPPORT FROM COLLABORATIVE NETWORKS

> Initially, I was supporting text applications and word processing, but then I moved into instructional technology. … I was very lucky, you know, given that I did not have a computer science or technical background, to be embedded in such a unit. One of those groups was running the academic Internet for Canada! … It was a very generative group. I was very lucky. (Geoffrey Rockwell, in Jacomy, 2023)

Most toolmakers are at risk of being isolated, if only because the needs of caring for technology shape their professional life in unusual ways. Collaboration with other toolmakers or technologists is a necessity, because they lack some of the competencies necessary to manage the full software stack of their tool, but also a source of joy and accomplishment, because it helps them progress towards the intellectual goals they have chosen.

It is not uncommon for toolmakers to develop partnerships with one another and with researchers. Tools are not simply products; they are also projects, and they have too many facets for a single person to cover comfortably: algorithm development and implementation, technical maintenance, system administration, user experience design, community management, fundraising, academic valorization … Many DCSSH toolmakers must find a way to be good enough at tasks considered entirely different jobs in the industry. That situation is similar to that of an artisan. For similar reasons, toolmakers may develop a form of companionship. They build up a network of partners through which they can access the competencies they lack, find mentorship, get emotional support from other toolmakers, and develop their craft. This network might be completely invisible to the researchers around them (when they work in a laboratory or research group) because it follows a different infrastructure specific to the toolmaker culture (e.g. GitHub, IRC channels).

The embeddedness of some toolmakers in such companionship matters to institutional responses to the lack of a dedicated career path, but also to the relations between toolmakers and the scholars using their tools. In a situation where a researcher makes a demand to a toolmaker, for example, to fix a bug or add a feature, the success of obtaining an answer largely depends on how the demand is presented to the toolmaker. Demands emanating from the companionship network are generally addressed quickly, as the toolmaker is in a situation of mutual dependency. If the demander is trusted by another toolmaker, they could act as an intermediary. Otherwise, just adopting the norms of the community will be appreciated, by filing the demand as an "issue" on GitHub (where the code of most DCSSH tool projects is hosted) rather than sending an email.

> It was very important for me and Stéfan that, year by year, we developed a partnership. It was not a one-off partnership. It was paper by paper, tool by tool, interface fiddle by interface fiddle. I do not think that the type of work we were doing can be done by one person. I could be a fairly good programmer, but Stéfan was better, and the two of us together could do more than apart. If I had had to do it on my own, or Stéfan had had to do it on his own, we would not have been as responsive. There would not have been the dialog, the issue, between the two of us. It was a blessing. This is just one of the lucks of life, that he and I were together at the same university at the right time, both with our different backgrounds, building things together. … I think that a lot more attention needs to be paid to the human elements of care. How do you enjoy working on a project with someone over time? (Geoffrey Rockwell, in Jacomy, 2023)

REFERENCES

ACLU (2024). Federal judge dismisses Elon Musk's X lawsuit against nonprofit researchers [Press release]. Retrieved March 25, 2025, from https://www.aclu.org/press-releases/federal-judge-dismisses-elon-musks-x-lawsuit-against-nonprofit-researchers

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. *Digital Research 2012* (pp. 1–3). The University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf

Barnes, N., Jones, D., Norvig, P., Neylon, C., Pollock, R., Jackson, J, Stodden, V., & Suber, P., (2013) *Science Code Manifesto.* http://web.archive.org/web/20130513204815/http://sciencecodemanifesto.org/ (archived 13/05/2013 and accessed 16/10/2025).

Boettiger, C. (2013).Doi != Citable. *Lab Notebook,* June 3. https://www.carlboettiger.info/2013/06/03/DOI-citable.html (accessed 19/06/2024).

Hutchinson, A. (2024). Meta abandons legal case over data scraping after losing key judgment. *Social Media Today.* https://www.socialmediatoday.com/news/meta-abandons-legal-case-data-scraping-losing-key-judgment/708538/ (accessed 13/08/2024).

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. *Reticular.* https://reticular.hypotheses.org/8039

Lamprecht, A.-L., Garcia, L., Kuzak, M., Martinez, C., Arcila, R., Martin Del Pico, E., Dominguez Del Angel, V., van de Sandt, S., Ison, J., Martinez, P. A., McQuilton, P., Valencia, A., Harrow, J., Psomopoulos, F., Gelpi, J. L., Chue Hong, N., Goble, C., & Capella-Gutierrez, S. (2020). Towards FAIR principles for research software. *Data Science*, *3*(1), 37–59. https://doi.org/10.3233/DS-190026

Latour, B. (1994). On technical mediation. *Common Knowledge*, *3*(2), 29–64.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. *Computational Communication Research*, *4*(2), 571–589.

Peixoto, T. P. (2023). *Descriptive vs. inferential community detection in networks: Pitfalls, myths and half-truths*. Cambridge University Press.

Perkel, J. M. (2021). Ten computer codes that transformed science. *Nature*, *589*(7842), 344–348. https://doi.org/10.1038/d41586-021-00075-2

Tenen, D. (2016). Blunt instrumentalism: On tools and methods. *Debates in the digital humanities*, *216,* 83–91.

22. Why do DCSSH toolmakers make tools?

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

We argue that DCSSH toolmakers attempt to design their tools to maintain alignment with their intellectual perspectives because the tools express their scientific agenda or even embody it when the method has been discovered through the toolmaking process.

The necessary tool ownership compels DCSSH toolmakers to prioritize specific design intents that reflect the constraints and incentives of their situation. Rather than creating universal solutions, they craft instruments with particular considerations in mind: making a technology easy to use, supporting learning through exploration, integrating with other existing tools, and remaining responsive to user needs while resisting feature bloat.

The chapter explores these design intents in detail, showing how toolmakers' specific situated perspectives shape the DCSSH apparatus. We emphasize that the tool design process is not unidirectional. While toolmakers guide tool development, researchers can also actively reshape tools through repurposing, contributing to user communities, collaborating with toolmakers, or even becoming toolmakers themselves.

DC TOOLS FOR SSH EMBODY THE TOOLMAKER'S SCIENTIFIC AGENDA

Every scientific instrument mediates and shapes knowledge production in specific ways, and thus possesses its own scientific agency. That agency emanates from the toolmaker's scientific agenda. From the toolmakers' perspective, toolmaking is an act of dissemination of their scientific perspective. That perspective does not need to be as precisely defined and formulated as in an academic paper, but it sometimes is. It is often purely methodological, but is sometimes more substantive. DC tools for SSH bear (infrastructured) theory, and their toolmakers are aware of it. To them, the freedom to design the tool they envision is the same thing as academic freedom for researchers, a way to express their own scientific agenda.

As we contended in Chapter 20 (Figure 20.1), an infrastructure becomes a tool when it ceases to be an invisible system and starts having an identity and a purpose: when it becomes a product. Product just means that everyone thinks and communicates about tools by naming them, associating them with a set of features, a purpose, and generally conceiving of them as whole things. Here we must underscore that although the concept of product is heavily ingrained in industrial project management, scientific instruments mostly come from a different place. Although they might occasionally become the point of a business, following FAIR principles, DC tools for SSH are often developed as open source software, or as part of a bigger research infrastructure. DC tools for SSH being products creates a need for specific resources (we will return to this topic in Chapter 24), and has other consequences for the

toolmakers. Products are intended for a certain public; products are designed by an authoritative figure (person or group); and products are curated.

The toolmaker is ultimately the one held accountable for the tool, with one important exception: tools big enough to be supported by a collective or an institution (i.e. with a large public, a clear identity, and relatively stable funding) are often designed by committee. This is the norm in large open source software projects, for instance, *OpenRefine* (Ham, 2013), whose development roadmap is decided by multiple actors in different ways, including an online forum, the leadership of the main maintainer, and a scientific committee. In this case, the collective is clearly structured and its accountability is distributed. Yet even *OpenRefine* started as "an open-source data cleansing tool named Freebase Gridworks… , conceived by David Huynh" in 2008 (OpenRefine, 2024). Most DC tools for SSH being quite small, they have a single or just a few toolmakers who must accept all of the accountability.

The toolmaker often has the capacity to make someone else's tool, but they will rarely do so. Some of us (authors) have been approached on multiple occasions by researchers who have a tool idea but lack the competencies to materialize it; most of the time, those researchers underestimate what it takes to make a tool. Those researchers seem to believe that once one possesses the skills, coding a simple tool only costs a moderate amount of time, which sounds plausible but turns out to be highly unrealistic. This is essentially because the costs of design, maintenance, and accountability remain hidden from them. However, in a research group where a toolmaker resides, those costs may become visible to fellow researchers, which paves the way for collaboration (which does not always come with academic attribution).

Here lies an important difference between a research software engineer (RSE) who does not happen to make tools and one who does: only the latter is led by circumstances to take on the role of "product owner" (to borrow an expression from the software industry). In that sense, many toolmakers are closer to the "Researcher-Developer" profile characterized by Baxter et al. (2012); although as we explained earlier, there are toolmakers of different kinds (see Chapter 20, Figure 20.1). We will rather call that profile the researcher-toolmaker to keep the emphasis on the matter of tools. The key point here is that a researcher-toolmaker is not as serviceable as an RSE, as they will only collaborate with a non-toolmaker insofar as the collaboration is low-friction and mutually beneficial. On the other hand, to the researcher, collaborating with a toolmaker ideally resembles closely collaborating with another researcher: on equal footing. In contrast, collaborating with an RSE is not only more asymmetrical (because the researcher drives the agenda) but also too often plagued with organizational overheads (because the RSE is shared between many projects). The serviceability of the RSE generates project management costs precisely because they are prevented from choosing their own agenda, while the researcher-toolmaker will collaborate as they see fit (insofar as they can).

One can easily picture that a researcher-toolmaker may desire to make a tool to embody and disseminate their methodological views. It is less easy to see that the tool also makes the toolmaker, notably by pushing them into the role of product owner, i.e. the author and designer of a piece of the scientific knowledge production process, which comes with accountability. These two movements reinforce each other and incite us (toolmakers and researchers alike) to identify the toolmaker with their tool; that is to reduce the toolmaker to a servant of the tool, while reducing the tool to an expression of its creator. Although there is generally some truth in this identification, it is also misleading.

Indeed, most toolmakers have other goals than attending to the needs of their tool (and its users); and tools are never transparent implementations of well-defined methods existing in

the toolmaker's mind. On the contrary, many tools can be seen as byproducts of scientific explorations where implementation and method are co-constitutive, invented together in the same movement. The scientific agenda asserted by the toolmaker via their tool does not necessarily preexist the tool (see Chapter 23). Some methods are invented by researchers and implemented by toolmakers, while others are discovered by the toolmaker themself through algorithmic experimentation that may leave a reusable piece of software that constitutes de facto a tool prototype.

Even when the toolmaker is "just" a discoverer of a method, even when their scientific agenda does not predate the making of the tool and derives entirely from it, the existence of the tool and its dissemination will assert a scientific agenda that will be attributed to them, to the extent that they are held accountable for the agency of their tool. It is very difficult for the toolmaker to distance themself from their tool; for example, to explain that what they advocate as a researcher may differ from what their tool does in certain contexts. For instance, the Gephi toolmakers have criticized in vain the use of community detection in some situations, even though the Gephi user community portrays it, in some tutorials and mutual help discussions, as an integral part of the tool's method (Jacomy & Jokubauskaitė, 2022). The almost inevitable identification of the tool with its maker prompts the latter to keep the tool as aligned as possible with their own scientific views and thus assert their authorship over it.

TOOLS ARE A TOOLMAKER'S PRIDE AND MOTIVATION

> I do not think anyone will ever read the books and the articles [I wrote] at some point in the future. … But I certainly am known for *Voyant*. … People have come up to me and said: Oh, I teach with *Voyant*! You know, nobody comes up to me and says: Oh, I read an article you wrote! (Geoffrey Rockwell, in Jacomy, 2023)

A key reward for assuming the role of toolmaker is to be valued and acknowledged by others. Yet surprisingly, that motivation rarely lies behind the initial act of creating a tool; it lies behind the acceptance of the purpose of sustaining one or more tools. One does not become a toolmaker by making a piece of technology, but by releasing and supporting it: a tool needs to last. The economy of toolmaking is dominated by the fact that withdrawing from sustaining the tool is easy and sometimes desirable, while staying with the trouble of making the tool last is costly. Toolmakers accept the burden of maintenance notably because they get different things in return.

The perspective of being associated with a piece of software may come with fears, such as being mocked for writing "crappy code that nobody likes", being exposed for bugs, or breaking the mysterious etiquette of open-source development (Kuzak, 2021; see also Jiménez et al., 2017). Some researchers who write code simply prefer to hide it because they see it as detrimental to their career (Yehudi, 2021). "The main reasons researchers give for this are embarrassment due to perceived poor quality code, a lack of confidence [that] the software is robust for other users and usages, and the time required to prepare it for release, including the provision of appropriate licensing and documentation" (Jay et al., 2021; see also Jay et al., 2016). Coding a piece of software does not make you a toolmaker insofar as you can simply deny it. Owning authorship of the tool is a premise of the toolmaker's figure that cannot be taken for granted. We will return in Chapter 24 to the problem of valuing toolmaking within academia.

On the other end, embracing the toolmaker's path may come with benefits. If the tool is popular, it may bring notoriety and recognition. If proxy papers (that get cited in place of the tool) are published or if the tool versions get a DOI, for instance in the *Journal of Open Source Software* (Smith et al., 2018), it may bring a wealth of academic citations. It may have currency in other spaces, from data journalism to the industry. It may unlock doors, albeit not necessarily the ones you want. And tools may have a bigger impact, a greater relevance than the papers you can afford to write. Your tools may outlast your writings and your ideas.

Yet even those benefits come with counterparts. We, authors of this piece, know the risk of becoming a "that-tool-guy", of getting enclosed in the purpose of caring for a tool while having broader ambitions and other skills. Inevitably, some people see us like that. This is specifically an issue for toolmakers on the academic side of the spectrum (Figure 21.1, chapter 21) who see their tools as just part of their output. Too often, making tools for research is not regarded as being as important as writing an argument in a text. There are so many research projects where authors have invested significant amounts of time in research and coding, only to be thanked in a footnote. Conversely, on the infrastructure side, an RSE who makes tools might find it desirable to have their toolmaking protected by their position's mission statement. Facing instrumentalization by users (researchers, but not only), and choosing whether to resist or not, is an integral part of the toolmaker's condition.

Hidden labor is another part of the toolmaker's condition. Repairing the tool when it breaks is an event, but maintenance and machine administration are not, and remain invisible (Denis & Pontille, 2022). As a result, the toolmaker often has to find ways to justify the time they spend on tasks which are invisible to other researchers, unless they simply do it during their free time. This is not specific to the DCSSH. From astronomy to zoology, virtually all great scientific findings of the modern age are enabled by powerful computers whose software is shaped by "scientist-coders" specifically capable of tackling the right research questions (Perkel, 2021); yet science institutions keep selling the heroic narrative of the super scientist whose theorization capabilities allow for a breakthrough to be found. A large part of the science toolmaking effort remains unaccounted for in current academic institutions. We will develop this point in Chapter 24.

Most toolmakers are proud of the tools they make and have a certain love for them. Their tools are an extension, an embodiment of their scientific agenda. Pride is a necessary component of the demanding economy of technology care, without which the toolmaker abandons their creation (see also Rieder et al., 2024). And to be clear, love and pride often coexist with some amount of hatred and shame. Toolmakers sometimes wish they were free from the chains of care and accountability, but they accept it as part of themselves. One becomes a toolmaker by accepting to stay with the good and bad sides of that strange attachment to a piece of technology.

DCSSH toolmaking is situated, and it shapes the design of the tools. Because DCSSH toolmakers are who they are, because their situation is what it is, they design their tools in certain ways and not others. They will often design them for ease of use and learning-by-doing. They will avoid implementing features that overlap with what other similar tools already do and generally resist expanding the scope of their tool too much, despite pressure from some other users. In the rest of this chapter, we will detail where these different design intents come from and how they shape the DC tools for SSH.

THEY DESIGN EASY-TO-USE TOOLS

> Humanists do not do research every day of the week, or every week of the year. They are teaching, and then all of a sudden, they do a bunch of research. They need tools that they can pick up quickly, use intensely, put down and then not use for six months. (Geoffrey Rockwell, in Jacomy, 2023)

Toolmakers are the first users of their tools and dislike wasting time on unnecessary steps. They automate and optimize; they blackbox technology. They tailor the tools to themselves, even if not exclusively. When they teach, they also optimize for their students because it makes their own lives easier. When they collaborate with scholars who use their tool, they adapt it to be more accessible to that audience. They make tools to be used by themselves, people around them, and strangers. Ease of use is the expression of the toolmaker's belief that being accessible is a good thing.

Toolmakers are aware that ease of use entails trade-offs that, if pushed to the extreme, could defeat the purpose of the tool. They put a limit on how simple tools can be, should be, and that limit depends on the circumstances. But none of us, toolmakers who write this text, ever made a tool intentionally complicated to deter users from using it. Tools are meant to be as simple as they can afford, or in other words, as complicated as they need, but no more. Unfortunately, that goal is not necessarily met.

A tool can be difficult to use for many reasons. First, it may be buggy, behaving inconsistently, crashing, or being laggy or slow. Toolmakers generally try to avoid this if they can, provided they have the resources and competencies. Second, it may be difficult to parse visually, lacking contrast or organizational principles, or requiring too much menu diving, too many clicks, or too much scrolling to reach its main features. These problems are due to poor graphic and/or interaction design, and although toolmakers want to mitigate them, they may very well misjudge the situation because they lack design skills or user feedback. Third, a tool may be confusing because it offers too many possibilities; the user feels overwhelmed and does not see where to take the actions necessary to accomplish their task. This misalignment between the maker and the user may be a design issue, but often also a simple public mismatch. Some tools are meant for experts who desire the many features they master, to the detriment of the beginner who needs to acquire that mastery. For toolmakers, this situation raises the question of how tools help beginners progress, if at all (Jacomy & Munk, 2024). Fourth, a tool may be hard to use because the choices that the user must make are difficult, as making those decisions entails an effort that the tool does not alleviate. This kind of difficulty is real but relative to the user's expectation that the tool assists them more than it actually does. The user may feel unsupported by the tool, yet the toolmaker may have excellent reasons to design it that way. The main reason is that the tool's method requires human judgment, because some decisions depend on factors outside the scope of the tool, or because the toolmaker considers that no algorithm can do a good enough job.

DC tools for SSH are meant to provide interpretive insights, not just produce output. Ease of use can be counterproductive when it allows the user to obtain results that depend on decisions they cannot make themselves, because they do not see what the choices entail, or even worse, because they are not even aware that those decisions exist. In such a situation, the user is not mastering the method supported by the tool. Toolmakers will sometimes limit ease of use when they consider it necessary to expose some decisions to the user, either for them to understand what is going on, or because those decision points are methodological commitments.

In some ways, toolmakers protect users from themselves, from their desire to obtain results effortlessly; but in other ways, paradoxically, toolmakers also wish that users just had a good time with the tools.

> We do not think that our users have a lot of patience. There are certainly people who will spend an hour installing Anaconda, and then one thing or another, and then play around with the tools, and read the books … And a week later they get something. But our tool is meant to be an entry level. You can either use a text we have already put up there, or you can paste some text in, or you can paste in a URL, or you can upload a text, whatever you can, and we will try to deal with it. … We are trying to make it as easy as possible for you to get started and play. (Geoffrey Rockwell, Jacomy, 2023)

THEY DESIGN FOR PLAY AND LEARNING-BY-DOING

> I would certainly agree that people should understand [a visualization they made with a tool]. But I think that there is an intuitive way in which humanists can partly understand it by kicking the tires of the results. … [Stéfan's and my] view is that we want people to start playing, and they will start understanding through playing. That is our philosophy of dealing with this, rather than putting them in a wizard situation where you have to make a series of decisions, installations, and then only if you have been a really good boy and passed the test, do you get to see anything. (Geoffrey Rockwell, Jacomy, 2023)

Some tools are designed so that the user can learn by using the tool—learn how to use the tool, of course, but learn other things as well: that there are different ways to do one thing, different algorithms and methods; how a given algorithm works; what the different steps of the method are, or could be; that different processes give different results, what they are and what they entail; and more.

For the time-constrained scholar, the tool is often a means to an end. But research has other moments where the tool is more than that, where the researcher can reflect on how they use the tool, what the method entails, and what it means to their research. In that sense, academia is profoundly different from other contexts like marketing or consultancy work. The tool cannot remain a purely utilitarian assistance; equipped practices must be explored, scrutinized, challenged, and reshaped.

Learning-by-doing is, of course, messier than the orderly learning-by-reading that most teachers practice. With reading, one can at least pretend that knowledge is some quality that the text passes to the mind. One can assess students by measuring the fidelity of their knowledge to the lesson. But not so with using a tool, where the knowledge is implicit. Knowledge is simply more than that, as we all know. If we look at the practice of reading more closely, we must face the fact that even with encyclopedias, we neither read from start to end nor by relevant entries. We wander, we skim, we jump, and we go back. We browse. Stephen Ramsay (2010) calls it "screwing around", and opposes it to "search", as in a library and as in Google. Browsing is "a completely different activity. Because once you have programmatic access to the content of the library, screwing around suddenly becomes a far more illuminating and useful activity" (ibid, p. 6). Ramsay defends browsing as a research method as valid as search. "It's not a matter of replacing one with the other, as any librarian will tell you. It is rather to ask whether we are ready to accept surfing and stumbling—screwing around, broadly understood—as a research methodology" (ibid, p. 7). The "hermeneutics of screwing around" (ibid)

with tools bring not only the discovery of the tool itself but also repurposing and invention. It may bring to light things that not even the toolmaker is aware of.

Toolmakers design for learning-by-doing and play for two reasons. First, because the tool is their means of expression. They prefer conveying knowledge through the tool's affordances rather than documentation because if they did not, they would write text and not code in the first place. Toolmakers often think, or rather experience, the method through their tool. Second, toolmakers generally find joy in using and making tools (which does not exclude pain, to be clear). Toolmaking entails play because it comes with surprises, unexpected turns, and calls for creative solutions, which one may also find in DCSSH research. DCSSH toolmakers tend to appreciate the creative aspect of toolmaking and research and naturally support it in their tools. Many DC tools for SSH are playful for that reason. They remain intentionally open to non-utilitarian uses as valuable epistemic practices.

> I do not think that we would have gotten where we were if we were not getting joy out of it. And part of the joy was actually thinking about the play. In English, there is a relevant use of the word play. Let us say you have a knob, you can fiddle with it until it breaks. That sort of play, playing with things that do not quite work right. There is a play between theory and implementation, which was really fun. It is like a rollercoaster. You go way up into the theory; we would build this sort of theory, and most of my colleagues were not lucky enough to be able to then go swooping down, making it work, and then making it work for other people. That rollercoaster, for me, was about vertigo. Roger Caillois has that theory of play, and vertigo is one of its elements. (Geoffrey Rockwell, in Jacomy, 2023)

THEY DESIGN TOOLS AS PARTS OF AN ECOSYSTEM

Open source tools, including those used in the DCSSH, are developed for coexistence with other tools. It may come as a surprise to those who think of tools as competitors in a market, but the makers of similar open-source tools often cooperate. We, the authors of this text, certainly feel that we are all in the same boat. That boat is named Unfunded Maintenance and has no sails: we are all rowing.

The necessity to minimize the burden of maintenance creates a natural division of labor between the tools. Some toolmakers think: if someone else's tool has a feature I wish my tool had, I could send users to that other tool and have fewer things to maintain. This mindset entails building bridges across tools and fostering intercompatibility. DCSSH toolmakers design their tools for coexistence, not for taking over the market. For the open-source toolmaker, having their tool become the only one is not the dream but the nightmare.

This inclination for coexistence contributes to shaping DC tools for SSH. For instance, unlike commercial products, free tools refrain from having their own specific file format. They typically use inputs and outputs in universal formats such as CSV, JSON, or XML. They tend to reuse open standards as much as possible.

Any open-source tool depends on a myriad of others and sometimes is depended on. It embeds the toolmaker in a large cooperative network, largely parallel to the research community—a network of like-minded software developers. DCSSH toolmakers pay respect to their fellow toolmakers. They typically avoid encroaching on the scope of another tool or will ask permission (e.g. to reuse an algorithm). The design of DCSSH tools is largely shaped by

this cooperation, notably through a division of labor where toolmakers aim to create tools that complement each other. This is why, as a user, you can rarely afford to learn just a single tool for a given need.

> The initial TAPoR project, the text analysis portal, was an imperial project; and that was part of the problem. It was trying to say: we can do everything for everybody! That was obviously unsustainable. Or at least we could not figure out how to sustain it. Whereas saying: look, we have got this thing, it is free, it is open source! You can do what you want with it; you can take the code; we publish papers with our ideas: you can take the ideas or disagree with them. I think that, by and large, we avoided any sort of imperial conflicts. Since then, *Voyant* has received some awards, like the Zampolli award, which is really the top award in the digital humanities for a project. I think that it reflects the fact that it is appreciated in the community. I hope that it is not seen as silencing anybody. (Geoffrey Rockwell, Jacomy, 2023)

THEY DESIGN TOOLS THAT DEAL WITH USER DEMANDS

> There is always a wrinkle that they want. And this is the beauty of adding the notebook programming environment. Now I can say OK, *Voyant* does what it does, but you have a notebook programming environment. Get a grant, hire a JavaScript programmer, and you can add your own thing. And if it is cool enough, we will take it, and build it into the main tool. (Geoffrey Rockwell, Jacomy, 2023)

Toolmakers and users often want different things, but the dissent is rarely about how a feature should work. Rather, it is about what features should or should not be part of the tool. It is a matter of scope. Typically, a user demands a feature that is reasonable on its own, but is problematic for the toolmaker for other reasons. The blackboxing of the tool hides that complexity from users, which is generally a good thing, as it simplifies usage, but it also gives the false impression that building a tool consists of stacking features as if they were bricks. Users may be frustrated to be denied features they deem simple or necessary. For example, users have been demanding an undo feature in Gephi for more than a decade, and the toolmakers have been working on it for years; yet it is still not ready. The fact that the undo feature is ubiquitous may mislead non-developers about the difficulty of implementing it.

When a useful feature is not implemented, it is generally because it is too costly either to implement or to maintain, or because it cannot be implemented for other practical reasons. For example, the users of social media software like 4CAT or the YouTube Data Tools sometimes demand features relying on data that is not available or against the terms of service of the platforms. More rarely, a feature is considered counterproductive by the designer, who intentionally refuses to implement it. Different toolmakers have different perspectives on this. Some tools are very opinionated (Gephisto is a good example; Jacomy and Munk, 2024), while others are as methodologically inclusive as possible (e.g. *SciKit Learn*; Pedregosa et al., 2011). But toolmakers will at least agree on one aspect of this: feature creep, also known as "featuritis", is a pathology of aging tools where the accumulation of features becomes detrimental to maintenance and/or usage. Featuritis is difficult to avoid. Either the toolmaker abandons their tool, or they support it long enough to face the issue. Some toolmakers choose to heavily curate the features of their tools and keep them minimal to limit design complexity and maintenance load.

> There is a ton of functionality involved [in *Voyant*]. Every time people come to me and say: Can I do X? I say: Yes, you just go here, you do that … Whoa! I did not even know that was there! And to some extent, this is the problem of featuritis and feature creep of software in general. *Voyant* has been around long enough. We keep on adding features. (Geoffrey Rockwell, Jacomy, 2023)

TOOLS GET RESHAPED BY THEIR USERS

> [*Voyant*] is made for humanists. … Having said that, … I get emails from lawyers to use it. An email from the son of a doctor whose father uses it. It gets used by a lot of different people. (Geoffrey Rockwell, Jacomy, 2023)

Tools travel differently than papers. They contribute to building communities in a different way. Some tools have unorthodox user communities organized around a specific repurposing. Gephi, for example, has been reappropriated by a part of the community of consultants doing search engine optimization (SEO). They have their own (non-scientific) goals and have developed their own practices; they constitute an epistemic subculture (Knorr Cetina, 1999; see also Jacomy & Jokubauskaitė, 2022). The users are not always who toolmakers think. As a tool travels to distant places, it breaks free of the sphere of people that toolmakers know and design for. The variety of users is often a surprise and a source of awe to toolmakers, who then wonder: what do users actually do with my tool?

Users are not fully free from the tool's design, insofar as the tool constrains them, but they can break free from the design intent. Researchers (and other users) can, will, and should repurpose DCSSH tools, because those are always more than the toolmakers intended.

The idea of repurposing is at the heart of the tools developed at the Digital Methods Initiative (DMI) in Amsterdam, like 4CAT (Peeters and Hagen, 2022), DMI-TCAT (Borra and Rieder, 2014), Netvizz (Rieder, 2013) or the YouTube Data Tools (Rieder, 2015). They all stem from the realization that various media need their own methods of investigation, which led the DMI toolmakers to repurpose online platforms and engines. The director of DMI, Richard Rogers, "positions digital methods as research sensitive to the methods embedded in the medium, as opposed to applying existing methods from social sciences and humanities" (Weltevrede, 2016). This reappropriation of the digital public space ultimately led to the creation of many tools. The practices of these toolmakers show that technology, like social media APIs, is never captive to its design intents. APIs are tools to some and toolmaking material to others. The DMI tools are built on top of other tools repurposed to enable research.

When you repurpose a tool, you practice toolmaking in essence, even in the absence of any material intervention on the tool itself. You practice methodological innovation, a form of scientific experimentation among others. If the user community is proactive in publishing knowledge about a tool, then it contributes to shaping it. Toolmakers are co-designing tools with users because the latter have the potential to reinvent them completely. To put it another way, DCSSH tools not only lie in their source code but also in the cultural artifacts that contextualize them, from online discussions, tutorials and documentation to teaching materials, scientific papers, and data sprints.

As we have seen, toolmakers have their own goals and motivation for making the tools we use in DCSSH. Their perspectives are partial. They do not necessarily aim at the greater good, first and foremost because there is far from a single version of the "greater good" for DCSSH,

but also because they make tools for specific reasons (e.g. sharing their scientific views, understanding APIs, helping others) while being subject to many constraints (e.g. accountability, technical limitations, lack of resources for maintenance). There is no superior authority deciding which tools should be created or sustained to best address the needs of DCSSH, and there is no reason to assume that those needs are appropriately covered by the DC toolbox currently available to SSH researchers. The apparatus available to researchers in the human sciences is shaped by forces other than a logic of satisfying academic needs.

That being said, researchers have more power than they may assume over the ecosystem of DCSSH tools. They may repurpose tools and contribute to reshaping their methods via tutorials and teaching materials; they may budget the features they wish for in grant applications; they may cooperate with toolmakers to reshape their apparatus; and they may engage with software and potentially become toolmakers themselves. In those different ways, they have the possibility to bring new design intents to the table and steer DCSSH tool development in other directions.

REFERENCES

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. *Digital Research 2012* (pp. 1–3). The University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf

Borra, E., & Rieder, B. (2014). Programmed method: Developing a toolset for capturing and analyzing tweets. *Aslib Journal of Information Management*, *66*(3), 262–278.

Denis, J., & Pontille, D. (2022). *Le soin des choses: Politiques de la maintenance*. La Découverte.

Ham, K. (2013). OpenRefine (version 2.5). http://openrefine.org. Free, open-source tool for cleaning and transforming data. *Journal of the Medical Library Association*, *101*(3), 233.

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. *Reticular*. https://reticular.hypotheses.org/8039

Jacomy, M., & Jokubauskaitė, E. (2022). Unblackboxing Gephi: How a user culture shapes its scientific instrument. Working paper. https://hal.science/hal-03729114/file/Unblackboxing%20Gephi%20Rev%201.pdf (accessed 26/08/2024).

Jacomy, M., & Munk, A. K. (2024). Interfering with the black-box-tradeoff model: Gephisto, a one-click Gephi for critical technical practice. *Convergence*, *30*(1), 142–166. https://doi.org/10.1177/13548565221129053

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. *International Journal of Digital Curation*, *16*(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Jay, C., Sanyour, R., & Haines, R. (2016). "Not everyone can use Git": Research Software Engineers' recommendations for scientist-centred software support (and what researchers really think of them). *Journal of Open Research Software*.

Knorr Cetina, K. (1999). *Epistemic cultures: How the sciences make knowledge*. Harvard University Press.

Kuzak, M. (2021). On the road to sustainable research software. *FOSDEM 2021*, communication in the Open Research devroom (video). https://archive.fosdem.org/2020/schedule/event/open_research_sustainable_soft/ (accessed 07/05/2024).

Jiménez, R. C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., ... & Crouch, S. (2017). Four simple recommendations to encourage best practices in research software. *F1000Research*, 6.

OpenRefine (2024). *OpenRefine Project History*. https://openrefine.org/openrefine_history (accessed 20/08/2024).

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M.,

& Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*(85), 2825–2830.

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. *Computational Communication Research*, *4*(2), 571–589.

Perkel, J. M. (2021). Ten computer codes that transformed science. *Nature*, *589*(7842), 344–348. https://doi.org/10.1038/d41586-021-00075-2

Ramsay, S. (2010). The hermeneutics of screwing around; or what you do with a million books. In K. Kee (Ed.), *Pastplay: Teaching and learning history with technology* (pp. 111–120). The University of Michigan Press.

Rieder, B. (2013). Studying Facebook via data extraction: the Netvizz application. In *Proceedings of the 5th annual ACM web science conference* (pp. 346–355). Association for Computing Machinery.

Rieder, B. (2015). YouTube Data Tools [software]. Available at: https://tools.digitalmethods.net/netvizz/youtube/ (accessed 09/08/2024).

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. *Convergence*, *30*(1), 216–235. https://doi.org/10.1177/13548565221127094

Smith, A. M., Niemeyer, K. E., Katz, D. S., Barba, L. A., Githinji, G., Gymrek, M., Huff, K. D., Madan, C. R., Mayes, A. C., Moerman, K. M., Prins, P., Ram, K., Rokem, A., Teal, T. K., Guimera, R. V., & Vanderplas, J. T. (2018). Journal of Open Source Software (JOSS): Design and first-year review. *PeerJ Computer Science, 4*, e147.

Weltevrede, E. J. T. (2016). *Repurposing digital methods: The research affordances of platforms and engines* [Doctoral dissertation, Universiteit van Amsterdam].

Yehudi, Y. (2021). Open research in life science: Funding foundational tools, trust, and talent. *FOSDEM 2021*, communication in the Open Research devroom (video). https://archive.fosdem.org/2021/schedule/event/open_research_life_science/ (accessed 07/05/2024).

23. The lifecyle of DC tools for SSH

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

We argue that DC tools for SSH are never finished products with a quiet life that can be anticipated by the toolmaker. Katz and Stall summarize the scientific tools' lifecycle as follows: "Research software is often developed by researchers themselves, often initially to solve a single problem, and then later generalized to solve other problems, and ideally shared so that other researchers can also benefit and avoid duplicate work required for development and maintenance. When this process works, the software can become community software, developed, supported, and maintained by a community of user-developers, expanding as it is applied to new problems. Whether or not this happens strongly depends on both community culture and associated incentives" (Katz & Stall, 2021).

We expand on those ideas by considering different kinds of tools in different situations. The topics we address follow tool lifecycles from start to end: how tools come into existence, how they get named, how their feature set evolves, how they get maintained, and how they stop working and die. Understanding these different stages of a tool's existence helps in thinking critically about the methods embedded in it, assessing how tried and tested they are, and how you could get involved in repurposing the tool and reshaping its scientific purpose. Indeed, many pieces of software that are not applications, such as scripts and online services, are also DC tools for SSH. Depending on your level of engagement with such objects, you may realize that you are already walking the path of toolmaking, or that you could contribute to improving or even just sustaining the tools you need for your research. We will notably take the time to discuss the necessity of maintaining DC tools and what it entails for the research community, a question critical to DCSSH toolmakers yet invisible to most scholars, who are not developers.

TOOLS COME IN DIFFERENT SHAPES AND SIZES

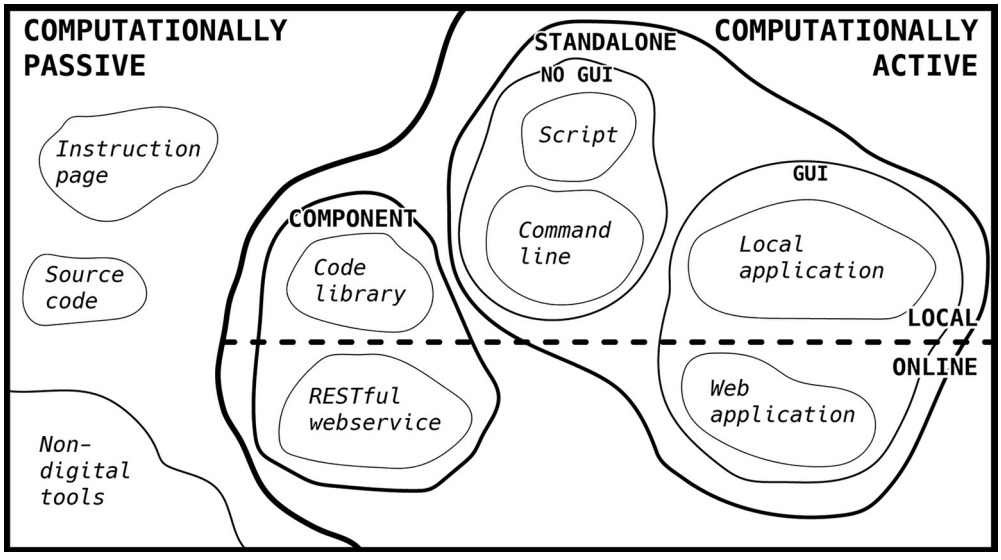
Some researchers have inventoried DC tools for SSH. The European Commission funded the SSHSOC project, which aimed "to provide a full-fledged Social Sciences and Humanities (SSH) Open Cloud where data, tools, and training are available and accessible for users of SSH data" (Veršić & Ausserhofer, 2019). It notably produced an online "open marketplace" (Social Sciences and Humanities Open Marketplace, 2023) where 2,165 tools are registered (as we write in August 2024). Those tools are categorized by function. Each tool may have multiple functions, and the 10 most represented are "analyzing" (548 tools); "data visualization" (344); "visual analysis" (283); "content analysis" (243); "discovering" (194); "annotating" (180); "capturing" (158); "disseminating" (151); "publishing" (148); and "collaborating" (144).

More interesting to us, the platform also classifies some of the tools by "invocation type" (SSH vocabulary service: Invocation type, 2023), i.e. according to how they present themselves materially to the user. Their ontology has only the following nine categories (in alphabetical order): "Command line"; "Command processor"; "Instruction page"; "Library"; "Local application"; "RESTful webservice"; "Script"; "Source code"; and "Web application" (ibid). Unfortunately, only a few of the entries seem to have this field filled, and it is not searchable in the online platform, which prevents us from counting which of those categories are the most represented. We can nevertheless use this ontology as a representation of the material diversity of DC tools for SSH.

We can organize those types by asking a few questions. The first question to ask: is the tool computationally passive or active? Computationally passive tools consist of pure inscriptions (knowledge, data), meant to be read or shared but not executed. The instruction page is of that kind. By extension, we could look at papers and books as tools meant to be read and enacted by non-computational means. The source code type is an edge case, as it represents the computationally passive mode of existence of an otherwise active device. Absent from the SSHSOC ontology (yet very real), some tools are materially active yet non-computational, like the ethnographer's pen and notebook, or recording devices such as microphones and cameras, whose size and appearance have a well-known impact on interviews, for instance. Non-computational tools are not the focus of this piece, but it is worth mentioning that many of our points about technology are not specific to the digital, such as the necessity to maintain an instrument, or the fact that it has scientific agency.

With computationally active tools for the SSH, there is a second question to ask: is the tool a component, or is it standalone? Components are tools for tinkerers and software engineers, but may look like infrastructures to non-coders. Those types are the library (e.g. *DaCy*, *SciKit-Learn*) and the RESTful web service (e.g. *Textgrid*; Neuroth et al., 2011). Researchers typically access such tools within *Jupyter Notebooks* (Kluyver et al., 2016) or from other tools like *Voyant* or *OpenRefine*. By contrast, standalone tools are self-contained and self-sufficient. We can differentiate those further by asking whether they offer a graphical user interface (GUI). The script and the command line tool are purely textual and have no GUI, the only difference between the two being that the script is meant to be read and edited while the command line is black-boxed (e.g. *Minet*); but any script can become a command line if needed. The local application (e.g. *VOSviewer*, *Gephi*) and web application (e.g. *DMI-TCAT*, *Hyphe*) benefit from a GUI, which is why we call them applications. Moreover, and transversal to those distinctions, we can also differentiate tools according to whether they run locally (on your own computer) or can only be accessed online (on a server, a cloud infrastructure, or via a web GUI). We summarized this landscape in Figure 23.1. With this description, we mean to showcase the diversity of what "tool" can mean in the DCSSH. Although we tend to think of tools as applications (standalone and with a GUI), many DC tools for SSH are in fact different.

Depending on your proficiency with the digital, the same device may look like an infrastructure or a tool to you. "Infrastructure is both relational and ecological—it means different things to different groups and it is part of the balance of action, tools, and the built environment, inseparable from them" (Star, 1999). Here, we define infrastructures as foundational support systems one can afford not to think about. As we already mentioned, a device can be considered a tool by someone and an infrastructure by someone else. *Jupyter Notebooks* can be at the same time an infrastructure to run computational methods and a tool to introduce students to data exploration. Similarly, a code library like *SpaCy* (Honnibal & Montani, 2017)



Notes: "Local" refers to software that can run on your own computer, while "online" refers to cloud-only systems. "GUI" stands for "graphical user interface."
Source: Author.

Figure 23.1 *Landscape of the different types of DC tools for SSH, looking at their materiality*

could be seen either as a dependency of another tool like *DaCy* (as part of its infrastructure) or as a tool on its own, to be drawn upon when certain needs arise. For similar reasons, some publics find command line interface (CLI) tools like *Minet* uncomfortable to use, while others find them superior to equivalent GUI-based applications: more straightforward, quick to use, and flexible.

Tools are rarely built from scratch. Most of the time, the main act of a new tool's creation is to integrate existing components into a more commoditized form, making them accessible to new audiences. Tools are made of other tools that are made of other tools, and so on; at every step, the contextual difference repurposes the components into something new. From an existing paper describing an algorithm, a toolmaker writes a script that implements that algorithm so that they can use it on their data. From an existing script, a toolmaker writes a library to easily reuse that method in different contexts. From existing libraries, a toolmaker makes a CLI tool to make it effortless to replay the method they always use. From existing libraries, a toolmaker builds a GUI-based application to make it easier for themselves and others to monitor the behavior of an algorithm and make methodological decisions. The seemingly minimal act of taking a piece of technology and moving it to a different context with a different audience is enough to make a tool. In the eyes of the new audience, the device is a tool because to them it makes a difference in how they access technology.

As tools age, they become more or less famous, more or less popular. Some tools are niche and speak only to specialists, while others are for everyone. Even so, it is not easy to guess if

a tool will become popular. Popularity is largely outside of the control of the toolmaker, who may wish their tool were more popular, but also sometimes less. Indeed, an audience growing by orders of magnitude changes a tool's conditions of existence, leading to the transformation of it, the social organization around it, and sometimes the toolmaker's career and personal life. From inception to oblivion, a tool's lifecycle may have many turns. We will now look into it.

MOST DC TOOLS FOR SSH ARE BORN FROM PRACTICE

> Stéfan and I began an agile praxis where the two of us would get together in a room, and we would try to do a project one day. … The goal there was to do a lot of projects and see what tools were useful, what was not useful. We just cobbled together what we had, and when we got something that worked, we would implement it for *Voyant*. (Geoffrey Rockwell, Jacomy, 2023)

One of the specificities of software as a technological environment is the flexibility offered by the possibility to duplicate code at virtually no cost. As the process of prototyping a tool entails iterating over implementations, the implementation cost will set the cost of the design process. Physical tools are costly to implement because each prototype is costly; software tools are not. There is still a cost to the tool design because it takes time and effort, but iterating once more is never an issue. While the economy of material tool design is constrained by the necessity to minimize the number of prototypes built, software design is driven by the opportunity to iterate at no cost. As a consequence, the iterations may be so tiny and frequent that the design process looks continuous. Some software projects may even embrace so-called continuous integration, where even the smallest (test-validated) code update is automatically built into a new published version. This has, in turn, two consequences for software instruments specifically: every prototype is a tool; and every tool is a prototype.

Every prototype is a tool in the sense that any prototype that is working, i.e. that makes a difference in a science context, can potentially be reused as a tool without adding anything to it. This is notably due to the fact that laboratories (in a broad sense) are places for experimentation where the apparatus is not necessarily black-boxed and polished, but semi-open and tinkered with. In other terms, researchers are particularly prone to sacrifice a lot of convenience if a tool solves a problem that is blocking their research. Of course, one is generally drawn to technology because it is powerful. Most SSH researchers, who do not define themselves as "digital" or "computational", have had or will have to face the discomfort of unknown software because their research calls for it. Typically, from the researcher's standpoint, if their colleague's half-baked *Jupyter notebook* does something that actually advances their own research, they will try to use it. Many unfinished pieces of software started circulating because they were useful. As van Es et al. (2021) observe, DC tools for SSH come first into being as "buggy prototypes [and] tentative solutions".

Every tool is a prototype in the sense that it is never finished. For example, *Gephi* is more than a decade old and yet has still not reached version 1.0, the milestone that its toolmakers reserve as a way to signify that it would be, in their eyes, finished. This is not specific to science but pervasive in the software industry. Anyone using a computer for work is probably familiar with the ever-evolving state of office software, which at times inflicts a surprising and frustrating amount of unfinishedness. But no piece of software is ever truly finished anyway,

as the technological context changes and requires any code to be maintained so that it keeps working.

Being able to update their tools is integral to toolmakers' condition. Indeed, the accountability they engage in with their tools needs to be mitigated or counterbalanced by other factors, notably the possibility of fixing their mistakes. As we mentioned earlier, some researchers are afraid to share their code (Jiménez et al., 2017; Kuzak, 2021), to the point that some even hide the fact that they code at all (Yehudi, 2021). Platforms like GitHub allow toolmakers to sustain a lively relationship with their users, updating their code and documentation, receiving and answering feedback, etc. Because tools are never really finished, toolmakers need a space to contextualize them and negotiate their accountability. This may mean, for instance, acknowledging that a tool has a bug or issue, but that the toolmakers do not have the time or resources to address it.

Every prototype being a tool and every tool being a prototype reinforce each other. Prototypes becoming successful do not incentivize toolmakers to invest in black-boxing them into finished products. Additionally, unfinishedness becoming the norm makes it easier to publicize a prototype early on.

Most DC tools for SSH are born from practice, not because tools of other origins have a lesser chance of success, but because any software project is grown continuously from a minimal piece. In the context of the DCSSH, that minimal piece is often a tinkering for a scientific experiment. The tinkering becomes a tool because it meets a public, which forces toolmakers to emerge as authors and manage their relation to the public. This forces tools to acquire an identity, a purpose, clearer boundaries, and to start becoming (never finished) products.

NAMING & IDENTITY MANAGEMENT IN TOOLMAKING

> At that time it was not yet called Voyant but Voyeur. At a certain point we realized that the word 'voyeur' in English has no positive connotations. In French, it is okay. A little bit better anyway. But in English … So we switched to Voyant. Some good friends of ours actually politely told us: this is not the right word for what you are doing. We love it, but it is not the right word. (Geoffrey Rockwell, Jacomy, 2023)

Making a tool entails committing to a name, a visual identity, a website, and many other things that weave the relationship to the public. This sometimes comes as a surprise to the toolmaker when the need to communicate about an otherwise unnamed piece of software emerges, for instance, when they decide to write a paper about it. The surprise may come another way, perhaps more widespread, where the tool has a temporary name and the toolmaker realizes, too late, that the emerging public is already referring to the tool that way. The toolmaker must then ponder whether to change the name or not.

The tool's name matters for multiple reasons. Of course, the name serves the purpose of communication, and it may be more or less apt at conveying the right message. But a name can be wrong for more practical reasons. The most important issue might be that the name already refers to another tool, sometimes similar. This is more common than one may think in science because the toolmaker is often incentivized by academic culture to remain descriptive and factual, and someone else had the exact same thought process elsewhere. Similar but different, if the name is a common word, it will prevent people from finding it easily in a

web search engine, like, for instance, Eytan Adar's *GUESS* (Adar, 2006). Other minor issues include internationalization, culturally sensitive terms, etc. Choosing an optimal name is not very important for a DCSSH tool, but avoiding unnecessary complications is valuable to the toolmaker, one of those complications being changing the name too late, which may entail rewriting a number of documents, migrating entire websites, and making the academic papers citing the old name look obsolete. For example, *Jupyter Notebooks* (Kluyver et al., 2016) was initially called *IPython Notebooks* but "was spun off from IPython in 2014 by Fernando Pérez and Brian Granger" (Wikipedia contributors, 2024; see also Pérez & Granger, 2007).

Gephi was initially prototyped by Jacomy as "Graphiltre", the contraction of "graph" and "filter", as it was about visualizing graphs and filtering them at the same time. It was subsequently passed on to Bastian, who redeveloped it as a more robust piece of software. At that point in time, it started meeting a broad public, organizing the project's communication became a necessity, and it was decided to find a better, more definitive name. That step was also a way for the team to reappropriate the project collectively. It was decided to keep the concept but simplify it: "graph" became summarized as the letter G, and "filter" as the Greek letter Phi, which also constitutes the tool's logo to this day. The team being French, they would pronounce those two letters "jé" and "fee" and thus write them as "Géphi." The accent would be dropped later on to avoid confusing an international audience. The English speakers discovering the tool would not know how to pronounce it, because its documentation was essentially in text form at the time, and a different pronunciation would emerge where the G is pronounced as in "graph." *Gephi* is still pronounced differently in France than in other countries to this day.

Identity is not necessarily easy to decide, but the hardest question might be who gets to decide the identity, when, and how. Setting the identity is a way, perhaps the main way, to claim authorship. This is due to how pieces of software come into being. Because the process is gradual, there is no clear starting point. And because it always entails existing code, the start always looks like a mundane act of bringing existing things together. Toolmakers don't have a clear childbirth moment; but they have a birth certificate moment, and it is to set (or reset) the tool's identity. Changing a tool's identity performs a reset of authorship because it is the point in its development that will be relevant to set the new tool apart from the old one, for instance, to differentiate *Jupyter* from *IPython*. Setting a tool's identity is an instrument of power for multiple actors at stake.

The toolmaker may have to deal with at least one other potential actor: their academic institution or their company. If the tool was built during work hours, does it belong to the university? If it was funded by a project, does it belong to its funder? As a tool becomes popular, new questions emerge: how to fund the project, how to maintain it, and in which direction to take it. If the toolmaker is not aligned with their hosting institution, painful problems may arise. Conversely, the toolmaker may proactively leverage their tool's identity to enroll other actors. For instance, the "DMI" in DMI-TCAT refers to the Digital Methods Initiative, the research group where it was developed. The *OpenRefine* name has a similar history. Initially created under the name Freebase Gridworks, it was "acquired" by Google (OpenRefine, 2024) and "the tool was renamed 'Google Refine'" (ibid). Then "Google transferred leadership of the project to the open source community, who renamed it 'OpenRefine' and moved it to GitHub" (ibid). The presence of "Google" in the name was a token of ownership that had to be subsequently removed.

If some toolmakers struggle to change the name of the tool, others struggle to keep it. This is notably the case when a tool changes and outgrows its initial scope or concept. For instance, *4CAT* (Peeters and Hagen, 2022) was made for analyzing the web board 4chan, but has since become a Swiss Army knife for data sources and analyses. *4CAT* used to mean "4chan Capture and Analysis Toolkit" (e.g. as quoted in Rogers, 2023), following the naming scheme of *DMI-TCAT*, where the first T stands for Twitter. *4CAT* subsequently opened up to several other online platforms, and the name became potentially misleading. The toolmakers decided to keep the name but to reinterpret its meaning. The title currently featured by the tool is "4CAT: Capture and Analysis Toolkit" (4CAT, 2018) and its logo is four cats, a way to offer a new justification for the 4 in the name.

The naming question shows that toolmakers are often drawn to become product managers, sometimes against their will. Identity politics do not stop at the symbolic life of the tool, however. They also permeate its functionalities and scientific content, as decisions have to be made about its scope and feature set.

ADDING AND REMOVING FEATURES

> *Voyant* does not do some things that I would like it to do. It is a moving target, especially when you are trying to make something that is, in some sense, current. (Geoffrey Rockwell, Jacomy, 2023)

The life of a tool is notably paced by the addition, and sometimes the removal, of features. What the tool can do largely decides how it must be built and who will use it. It is an editorial choice made by the toolmakers, and in the case of DC tools for SSH, a series of methodological commitments as well. The tool must change under the double influence of the evolution of scientific knowledge and of the toolmakers' own scientific views.

The editorial choices about the feature set are constrained by conflicting influences. The toolmaker may want to follow a new state of the art but lack the time, or the skills. The toolmaker may also intentionally disregard the scientific consensus to the benefit of their own method, if they see their tool as a vessel for their own academic influence. And the toolmaker may have to make choices for practical reasons, regardless of the scientific value of some features: because some features depend on other features, or are incompatible with other features; or because they make the maintenance easier, or more difficult. For example, the authors of *Gephi* removed multiple features, like the support for hierarchical graphs, from version 0.9.0 in 2015 (Gephi team, 2022), because they made the maintenance too costly. Another common situation is the addition of a feature for the sole reason that it was promised in an academic grant. A large part of the tool life will be decided by the toolmaker's negotiation of the feature set, governed both by the necessity to support the project through funding and maintenance, and by the desire to shape the tool in a certain way.

Here is another example from the *Gephi* project: the undo feature. Users make mistakes, and in virtually every mundane piece of software, the undo feature allows them to go back to the state of their work before the mistake. That feature is standard and ubiquitous, yet it is absent from *Gephi*. Users have been asking for it for as long as it has existed. The toolmakers are aware of it, but the feature is simply too hard to implement to make it to the top of the priorities list, given the time they have to dedicate to the project. The situation changed when a group of scholars from the Netherlands worked with a developer to develop the Fieldnotes

Plugin (Wieringa et al., 2019), an extension to *Gephi* allowing the user to automatically log their activity within the tool into field notes documenting their network analysis process. As it turns out, building an undo feature similarly requires keeping track of every user action. Some tools even merge the activity log and the undo feature together, for instance, *OpenRefine*. During the annual *Gephi* coding meeting in 2021, the toolmakers worked on that feature with the *OpenRefine* maintainer Antonin Delpeuch. In their report, they write: "We learned how [the undo/redo feature] is done currently in OpenRefine, the problems it creates, and how it could be done better. The feature is doable in Gephi, and we sketched an architecture" (Gephi team, 2021). The *Gephi* project benefited from interesting feedback from another similar project. As we write (summer 2024), the undo feature is still in progress. The 2021 working session helped break the feature down into smaller sub-projects that can be implemented one after another.

THE BURDEN OF MAINTENANCE

> At a certain point, we realized that [our current] model was too unwieldy. We would have to get $1,000,000 a year to keep development, and this was not going to be sustainable with the type of funding you get in the humanities. So we … broke it up into smaller pieces that we could support. … And at a certain point, we realized: this is very popular! We do not have the problem of nobody using it anymore. Now we have got the problem of how to maintain it. (Geoffrey Rockwell, Jacomy, 2023)

Any technology decays without maintenance, even the digital. "Software rots over time. Not because the code itself deteriorates, but because the environment it runs in inevitably changes" (Rieder et al., 2024). Yet because maintenance is invisible, one may think of it as a daily routine where the maintainer checks that the status quo is preserved. Unfortunately, this is almost never the case. Maintenance comes in many shapes, and the maintainer has almost no say on what it will demand, how much it will cost, and when it will require intervention. Maintenance shapes the tool by constraining the toolmaker to make radical decisions or allow the tool to break and decay.

Maintenance is care under multiple forms, both as caring about and as taking care of. "As many commentators have observed, care … refers both to a mental disposition of concern and to actual practices that we engage in as a result of these concerns" (Tronto, 1998). Following Joan Tronto, Rieder et al. (2024) propose to look at the toolmaker through the angle of "ethics of care". Following Annemarie Mol (2008), we understand care as a set of practices undergoing a logic of interdependence, where the toolmaker must sometimes do what is needed rather than what is desired (by them, users, or scientific authorities). Tool design and maintenance entail care directed at the tool first but also, through it, at methods and various other scientific activities, the research community, other users, the toolmakers themselves, and their loved ones (laboratory, IT staff, fellow toolmakers …). Maintenance is directed towards attending to their needs to varying degrees.

Maintenance differs from repair in that it does not make an event. Repair can be dramatized as a heroic narrative, but maintenance is invisible (Denis & Pontille, 2022). Terry Gilliam's 1985 movie *Brazil* features the central heroic character of Archibald "Harry" Tuttle, a freelance heating engineer who illegally repairs people's air conditioning systems in place of incompetent Central Service workers, leading to him being accused of terrorism. Tuttle is

heroic because his intervention makes a visible difference; it returns the system's state from broken to functional. By contrast, maintenance prevents the system from breaking, which has no apparent effect. Like Tuttle, the repairman comes from the outside, fixes, and leaves; he constitutes outside assistance. The maintainer lives in contact with the maintained thing and is taken for granted as if it were part of it. Maintenance does not have heroes.

Maintenance cannot have heroes because it doesn't let itself be dramatized in this way. Although this comes with problems, it is not necessarily an issue for toolmakers. Indeed, toolmakers are not outsiders bursting in to fix things with their superior powers. Rather, if indeed they are researcher-programmers, they live in close proximity to the situation in which their tool operates, which is why they could make it in the first place.

The invisibility of maintenance is not a problem in itself, and toolmakers do not necessarily wish to be heroes, but it still comes with a major concern: maintenance is very difficult to fund. Academic institutions agree to fund a new feature or the fixing of an issue, but not the invisible work of preventing tools from breaking in the first place. We will develop this point in Chapter 24. Letting a tool break before fixing it may be productive for the toolmaker, as it reminds the public about the necessity of caring for the tool. This may allow them to be dramatized heroically, but at the same time, the breaking also points to a failure of maintenance and may lead the tool's audience to distrust it. From the toolmaker's standpoint, the sustainable solution lies in convincing funding bodies to fund tool maintenance despite its lack of visibility, which is very difficult to achieve.

Gephi underwent a massive, long-lasting failure from 2013 to 2015. The problem arose from recent (at the time) versions of macOS requiring a Java update that was incompatible with *Gephi*'s core, a critical part of the tool. *Gephi* would just refuse to run. The core could not be fixed easily for a specific reason: the developers' energy was focused on rewriting the core entirely. As the toolmakers explain, "in March 2013 the time had come to think about what Gephi 1.0 would look like and realize it needed a new core. This was by far the most complex project the team had to overcome but developers had a long-term vision and [knew] that future developments will now rely on a robust and extensible core, with world-class performances" (Gephi team, 2015). Fixing the old core would not only take time and effort, but also impair the advancement of the tool and postpone the release of the new version. The team therefore decided to focus on releasing the new version as soon as possible. Unfortunately, releasing the new engine took about two years, during which the situation got worse. In early 2013, only a few students with the very latest release of macOS had an issue with *Gephi*, and it could pass as an exception, a temporary bug. But in late 2015, most students with a Mac encountered the incompatibility issue. This sparked the rumor that *Gephi*'s development had been abandoned, and some teachers who used that tool started switching their courses to alternatives. The release of version 0.9 in December 2015 fixed the issue and brought other improvements, allowing the public to trust the project again, but maybe with more awareness of its precarity. It also led the toolmakers to rethink the conditions of their own resilience. In 2018, they wrote: "we cannot just produce a version of Gephi that works well and let it be. It would stop working because technology changes. … Sadly, when incompatibilities arise, it is generally for the core developers to deal with. … We are not sure we will be able to keep up efficiently with these changes because of our limited technical leadership" (Gephi team, 2018a). The team has since been working on stabilizing maintainers for the project.

As Denis and Pontille (2022) highlight, the maintained thing is not that which does not break, but that which lets itself be maintained. A tool's maintainability depends on the clarity

of its code, the availability of the developers familiar with that code, and the flexibility of the software development infrastructure. It also depends on the maintainers' ability to attend to the needs of the tool, i.e. to detect issues before they grow into breaking bugs that impact users. The most typical of those needs is to have software dependencies up to date; but also, sometimes, to not update to the latest release if it is not stable enough. The toolmakers often work on a version of the tool that is more recent than the latest release, but only publish it when they have validated its stability. Quite often, they use an automated test system to check that code interventions do not break the preexisting features; they have more insights about the codebase than a regular user. If they use a crash reporting system, they also get notified when a new problem emerges on user machines. This allows them to continuously attend to the fragility of the tool.

Many DC tools for SSH are precarious, notably because the toolmakers are not provided with the resources to prevent problems from arising. As a consequence, maintenance "is also 'choppy' and requires intermittent and difficult to anticipate intervention, often with a high level of urgency" (Rieder et al., 2024). Breakdowns and repairs are not just an annoying part of DCSSH tool maintenance; they constitute opportunities for change that pace the tool's evolution. The nature of maintenance has an impact on the methods available to SSH scholars.

Indeed, there is always an ontological aspect to repair and maintenance (Denis and Pontille, 2022). The maintained tool is never left unaltered. This goes against the widespread notion among DCSSH scholars that their apparatus somewhat consists of finished products, thoughtfully released because their feature sets reflect toolmakers' intentions. This understanding ignores the material resistance of the computer environment, hardware and software alike. Not only is the tool design constrained, but every act of maintenance may impose a transformation. Repairing and maintaining is never really about returning to the unbroken state. For toolmakers, there is no status quo; there are no stable criteria.

Here is yet another example from the *Gephi* project. Its graphic engine relies on a code library (JOGL) that gradually stopped being maintained around 2021. It started introducing bugs for some users that the *Gephi* toolmakers could not address because they were internal to that library. It was impossible to replace that library without rewriting the graphic engine, but there was no other solution. The toolmakers started rewriting the engine, which offered two opportunities: using more recent techniques to improve performance and architecturing the code to make it easier to swap libraries in the future. Rewriting the engine took a few years, and it became usable in 2024. In a strange turn of fate, the problematic library became maintained again during that time, and the *Gephi* team ultimately decided to keep using it in the new engine. This entire endeavor can be seen as a maintenance operation, aiming to prevent *Gephi* from breaking because of a broken code dependency. But it did not simply preserve the tool's status quo; it changed it with a sensible impact for both the toolmakers and the users, hopefully for the better.

TOOLS DO NOT LAST FOREVER

> I have this suspicion that the days of *Voyant* may be coming to an end, that it might be the return of the command line or the prompt line. That will become the new paradigm for data analytics. And it will not just be *Voyant*, it will be *Tableau*, it will be all sorts of tools that get replaced by these new tools [using generative AI]. (Geoffrey Rockwell, in Jacomy, 2023)

Tools do not last forever, if only because they require maintenance. Code may remain unaltered, but the rest of the ecosystem keeps evolving; thus, "software rots over time" (Rieder et al., 2024). The digital infrastructure slowly changes, and without care, software decays.

Some kinds of tools can be more resilient than others because some parts of the infrastructure are more stable than others. From the toolmakers' standpoint, using lots of libraries is preferable to reinventing the wheel, but it puts them in a difficult position as well. Dependencies are vulnerabilities, as we have seen. Libraries also make the installation process more difficult, notably when specific versions of those libraries are required (and each library depends on other libraries, and so on). Some tools are built on top of platforms that alleviate incompatibility problems in various ways. Java (used by *Gephi*, *VOSviewer*, and *OpenRefine*) promises a multiplatform deployment (macOS, Windows, Linux …). Docker (used by *Hyphe*) promises complete control over the package environment. The web as an application platform (*4CAT*, *DMI-TCAT*, *YouTube Data Tools*, *The Lippmannian Device* …) promises universal access from the browser (a promise not necessarily kept). Those resilience strategies, and others, always come with a cost in code complexity. Alleviating the burden of maintenance by making the tool more resilient to breakdowns also makes the code more difficult to reappropriate by other developers. The software industry has developed standards and good practices to ensure that the code can be reappropriated, but only professionally trained developers master them. A research software engineer (RSE) is more likely to possess such skills than a self-taught researcher-toolmaker. The idiosyncrasy of many DC tools' codebases creates a dependency on the toolmaker. Because of this, even if the source code is open, the toolmaker may not be able to pass their tool on to other people, or at least not without significant effort.

As a matter of fact, tools rarely survive the loss of their toolmakers. Passing on a tool to others is hard because toolmaking is mostly a duty. Unless the toolmaker succeeds in enrolling others to reappropriate and own the project after them, when they abandon the tool, it decays and becomes unusable; it dies.

> To be honest, right now, I am nearing retirement. My feelings right now is that I want to find a way to gracefully pass this on to someone else. … I have put a lot of work and I have gotten some funding, so I have probably enough money for five years, and I am going to use those five years to pass it on; or let it die. (Geoffrey Rockwell, Jacomy, 2023)

OPEN SOURCE DOES NOT GRANT SUSTAINABILITY

The question of software decay is transversal to the entire open-source software movement. Although the open-source infrastructure allows sharing and passing the code on, it does not guarantee that someone will take the project from there and bring it further. There is no crowd of maintainers out there waiting for free code to maintain.

Scholars unfamiliar with the open-source community may have the wrong expectations. The open-source infrastructure—for instance, GitHub—is a necessity for sharing a project; but it is not sufficient. Most of the time, the coders use it to share their work within a community that is already established. Although having such open infrastructure helps find new contributors, most of the recruiting work has to happen in other places, like conferences and workshops.

> 'Open-sourcing' software is sometimes suggested as a solution [to the maintenance problem], from the assumption that others will contribute or continue the work as long as the project's source code and documentation are available. This passes over the substantial knowledge and experience gathered during the work on a tool, which feeds directly into further development. It also misses that well-functioning open source projects generally have strong and proactive leadership or community management. (Rieder et al., 2024)

For projects as niche as DC tools for SSH, the apparent promise of getting new contributors through the open source infrastructure is largely an illusion. Yet for the most popular projects, it sometimes happens, as a rare and precious event. The *Gephi* team has been trying to attract developers since the beginning of the project, and more proactively since 2021. No developer has joined the project core in years, for reasons that have been largely identified (Jacomy & Bastian, 2023): Java is an old technology that is not attractive to young developers; senior Java developers in the open-source community expect a financial income that the *Gephi* project cannot afford (so far); the researcher-toolmakers who have the time and good will to contribute have the competencies to create and maintain a plugin, but not the core of the software. The project's current strategy is to retain plugin developers and introduce them to the codebase of the core, so that as many people as possible have the knowledge to intervene if necessary. That being said, there is a major exception to this situation. *Gephi*'s current lead developer, Eduardo Ramos Ibáñez, was introduced to the team by applying in 2010 to the Google Summer of Code, an infrastructure set up by Google to sponsor a selection of open-source projects by funding students to work on the code during the summer break. Eduardo was not the student selected for the project, yet he stayed involved and kept contributing, even without funding. He designed significant parts of the tool over the years, notably the "data laboratory" (a spreadsheet view of the nodes and edges). He mastered the codebase of *Gephi*'s code and became the lead developer in 2018 (Gephi team, 2018b). The case of *Gephi* is an outlier, however. Many worthy open-source projects fail to attract a single developer.

By writing that "open source does not grant sustainability", we mean to emphasize two things. First, although sustainability is desirable, there are better ways to achieve it than open-sourcing the code. Those ways are compatible with open source, but in many ways independent. We will return to this in the next chapter. Second, the purpose of opening a tool's sources should not be to grant sustainability. It should be to allow methodological scrutiny, foster reproducibility, and defend code as a commons. The open-sourceness of many DC tools for SSH should follow from the open science movement. Open-sourcing code for the purpose of attracting free maintenance labor is not only naive; it is also doomed to fail and counterproductive. As all open source contributors know, onboarding a project takes much more than the ability to download the code. It is first and foremost a social process. Opening the source code has many benefits by itself (allowing scrutiny, facilitating teamwork, incentivizing good practices), but it does not suffice to make a tool sustainable.

> I only know of one team that has gone in and done something with the code, where they wanted to change something and then reverted back to us. (Geoffrey Rockwell, in Jacomy, 2023)

REFERENCES

4CAT (2018). OILab. https://4cat.oilab.nl/ (accessed 29/08/2024).

Adar, E. (2006). GUESS: a language and interface for graph exploration. In *Proceedings of the SIGCHI conference on Human Factors in computing systems* (pp. 791–800).

Denis, J., & Pontille, D. (2022). *Le soin des choses: Politiques de la maintenance*. La Découverte.

Gephi team (2015). Gephi 0.9 released: Play with network data again. *Gephi Blog*. https://gephi.wordpress.com/2015/12/21/gephi-0-9-released-play-with-network-data-again/ (accessed 30/08/2024).

Gephi team (2018a). Is Gephi obsolete? Situation and perspectives. *Gephi Blog*. https://gephi.wordpress.com/2018/11/01/is-gephi-obsolete-situation-and-perspectives/ (accessed 30/08/2024).

Gephi team (2018b). Meet Eduardo, our new lead developer. *Gephi Blog*. https://gephi.wordpress.com/2018/11/01/meet-eduardo-our-new-lead-developer/ (accessed 30/08/2024).

Gephi team (2021). Gephi code sustainability retreat 2021: Debriefing. *Gephi Blog*. https://gephi.wordpress.com/2021/12/06/gephi-code-sustainability-retreat-2021-debriefing/ (accessed 30/08/2024).

Gephi team (2022). *Releases*. GitHub. https://github.com/gephi/gephi/wiki/Releases#removed-features (accessed 29/08/2024).

Honnibal, M., & Montani, I. (2017). *spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing.*

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. *Reticular*. https://reticular.hypotheses.org/8039

Jacomy, M., & Bastian, M. (2023). Gephi towards v1.0: The codebase, and the rest. *FOSDEM 2023*. https://archive.fosdem.org/2023/schedule/event/graph_gephi_future/ (accessed 29/08/2024).

Jiménez, R. C., Kuzak, M., Alhamdoosh, M., Barker, M., Batut, B., Borg, M., Capella-Gutierrez, S., Hong, N. C., Cook, M., Corpas, M., Flannery, M., Garcia, L., L. I. Gelpí, J., Gladman, S., Goble, C., Ferreiro, M. G., Gonzalez-Beltran, A., Griffin, P. C., Grüning, B., ... & Crouch, S. (2017). Four simple recommendations to encourage best practices in research software. *F1000Research, 6*.

Katz, D. S., & Stall, S. (2021). Software citation. *Information Services & Use, 41*(1–2), 39–42. https://doi.org/10.3233/ISU-210108

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter Notebooks–a publishing format for reproducible computational workflows. In B. Schmidt & F. Loizides (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press.

Kuzak, M. (2021). On the road to sustainable research software. *FOSDEM 2021*, communication in the Open Research devroom (video). https://archive.fosdem.org/2020/schedule/event/open_research_sustainable_soft/ (accessed 07/05/2024).

Mol, A. (2008). *The logic of care: Health and the problem of patient choice*. Routledge.

Neuroth, H., Lohmeier, F., & Smith, K. M. (2011). Textgrid–virtual research environment for the humanities. *International Journal of Digital Curation*, *6*(2), 222–231.

OpenRefine (2024). *OpenRefine Project History*. https://openrefine.org/openrefine_history (accessed 20/08/2024).

Peeters, S., & Hagen, S. (2022). The 4CAT capture and analysis toolkit: A modular tool for transparent and traceable social media research. *Computational Communication Research*, *4*(2), 571–589.

Pérez, F., & Granger, B. E. (2007). IPython: A system for interactive scientific computing. *Computing in science & engineering*, *9*(3), 21–29.

Rieder, B., Peeters, S., & Borra, E. (2024). From tool to tool-making: Reflections on authorship in social media research software. *Convergence*, *30*(1), 216–235. https://doi.org/10.1177/13548565221127094

Rogers, R. (2023). *The propagation of misinformation in social media: A cross-platform analysis*. Amsterdam University Press.

Social Sciences and Humanities Open Marketplace (2023). Webpage. https://marketplace.sshopencloud.eu/ (accessed 27/08/2024).

SSH vocabulary service: Invocation type (2023). https://vocabs.sshopencloud.eu/vocabularies/invocation-type/invocationTypeScheme (accessed 2024-08-27).

Star, S. L. (1999). The ethnography of infrastructure. *American Behavioral Scientist*, *43*(3), 377–391.

Tronto, J. C. (1998). An ethic of care. *Generations: Journal of the American society on Aging*, *22*(3), 15–20.

van Es, K., Schäfer, M. T., Wieringa, M. (2021). Tool criticism and the computational turn. A "methodological moment" in media and communication studies. *Medien & Kommunikationswissenschaft*, *69*(1), 46–64.

Veršić, I. I., & Ausserhofer, J. (2019). Social sciences, humanities and their interoperability with the European Open Science Cloud: What is SSHOC? *Mitteilungen Der Vereinigung Österreichischer Bibliothekarinnen Und Bibliothekare, 72*(2), 383–391.

Wieringa, M., van Geenen, D., van Es, K., & van Nuss, J. (2019). The field notes plugin: Making network visualization in Gephi accountable. In A. Daly, K. Devitt, & M. Mann (Eds.), *Good Data*. *INC Theory on Demand*. Available at: http://networkcultures.org/blog/publication/tod-29-good-data/ (accessed 30/08/2024).

Wikipedia contributors (2024). Project Jupyter. *Wikipedia*. https://en.wikipedia.org/wiki/Project_Jupyter (accessed 29/08/2024).

Yehudi, Y. (2021). Open research in life science: Funding foundational tools, trust, and talent. *FOSDEM 2021*, communication in the Open Research devroom (video). https://archive.fosdem.org/2021/schedule/event/open_research_life_science/ (accessed 07/05/2024).

24. Valuing toolmaking in academia

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

This chapter addresses the complex relationship between toolmaking and academia in DCSSH. We examine how the academic system struggles to properly acknowledge and reward the labor of developing and maintaining research tools, despite their growing importance to scholarly work. The fundamental challenge lies in how tool work often fails to be recognized as academic achievement, creating a problem of hidden labor that affects both the creation and maintenance of research tools. We aim to provide advice to aspiring toolmakers navigating these challenges, while simultaneously raising awareness among tool users about the often invisible work that sustains our research infrastructure. Our hope is that by sensitizing tool users to the systemic challenges inherent in developing and maintaining tools within academia, we can collectively cultivate a better environment for DCSSH research.

The academic environment's resistance to toolmaking has prompted various adaptive strategies. We explore how toolmakers repurpose traditional academic mechanisms like publication and grant funding to legitimize their work, showing both the possibilities and limitations of these approaches. Yet the situation is not static—recent years have seen meaningful changes in how institutions value and support research software development, and we will also discuss these evolving trends and their potential impact on the future of toolmaking in academia.

OPERATIONS DO NOT PASS AS ACADEMIC

> Stéfan [Sinclair] and I had to move to a more professional, production-oriented approach. And of course, the academy does not reward production. I cannot sit there and say: we now have just released *Voyant* 2.3, and it might have been as much work as three articles. I got credit for *Voyant* and *Hermeneutica* and that was it. You do not get it again and again unless you do something dramatic. … This is one of the problems with us academics: we are not rewarded for production systems. (Geoffrey Rockwell, in Jacomy, 2023)

Schreibman and Hanlon (2010), in their survey of toolmaking in the digital humanities (DH), report that DH toolmakers are aware that their practice "is not necessarily considered scholarship in and of itself. This perception, held by the vast majority of tenure review boards, dissertation committees, and our peers, may be an impediment to the development of the field of digital humanities" (Schreibman & Hanlon, 2010).

Let us reuse here the classic project management dichotomy of project work versus operations work. The main problem of toolmaking in academia is not the project work; for instance building a prototype, because it happens once, makes an event, and as such can be valorized, typically via a peer-reviewed publication. The main problem is the operations work, like maintenance, that must be done again and again. The never-ending nature of that part of toolmaking is an issue for both fundraising and academic valorization.

Funding bodies typically seek to invest in self-sustaining projects, which essentially means minimizing operating costs. Tools have operating costs, especially when they become popular. These costs do not consist only of code maintenance or keeping up the infrastructure necessary for building and serving the tool, but also include managing the user community, maintaining the website, and the documentation, etc. These tasks are difficult to fund. Some, but not all, can be delegated to the user community; and the community needs a contact point (a project manager) to remain engaged.

Similarly, operations are typically considered out of the scope of academic publication for essentially two reasons: they do not (seem to) bring anything new, and their output is not considered knowledge. Most acts of maintenance will be seen as conservatory: preserving the status quo or reverting to it. The ontological share of maintenance is invisible when the change is distilled slowly over multiple mundane interventions. The academic battle for novelty can only be fought on the grounds of dramatic changes. But even when the toolmakers' work can be painted as innovative, it will too often be rejected during peer review for not consisting of abstract, reusable knowledge. Describing a new algorithm is publication-worthy, but implementing it in a new tool is not. Code is generally seen as the material production of a worker, not as the knowledge production of a thinker. In academia, coding is not typically conceived of as writing, although the situation is changing, as we will see.

> At my university, my annual report is going to be assessed by a bunch of chairs and colleagues in the humanities. I get along fine with them, and if I came out with a brand spanking new tool for that year, most of them would give me credit for it; but not for the next year. It would be just like a book: you get credit that year, not the next year. The fact that you are maintaining it does not count. (Geoffrey Rockwell, Jacomy, 2023)

ACADEMIC TOOLMAKING'S HIDDEN LABOR PROBLEM

At the heart of the DCSSH toolmaking economy lies the reality that it entails hidden labor: a sensible share of the time and effort necessary to bring and maintain tools into existence will remain hidden. Unfunded because invisible, that effort is often at the toolmakers' expense.

As Derrick et al. (2024) write about the Festival of Hidden REF (for Research Excellence Framework) they organize, "academic reward is blind to much work that is critical to research, and to roles that make this research possible". They call these "hidden roles", and those are not just toolmakers. "Some, like Librarians and Technicians, have existed for as long as research itself, while roles such as research software managers and administrators, Research Software Engineers and Data Managers, have arisen more recently (Kerridge et al., 2023). These 'hidden roles' have been overlooked by a culture that fails to look beyond publications as a means of identifying and rewarding value in research culture" (Derrick et al., 2024).

Smithies et al. (2019), who maintain DH projects at the King's Data Lab (KDL, a laboratory at King's College London), defend the necessity of making the problem visible. "Teams like KDL struggle with issues presented by myriad pressures: it is neither fair nor productive for Principal Investigators (PIs), funding agencies, and the wider community, to have the reality of those pressures hidden from them. Significantly, rather than seeing such issues as embarrassments, to be hidden from administrators, funders, and colleagues inside and outside our

institution, the lab recognises them as research opportunities for developing enhanced methodologies" (ibid).

The toolmaker's hidden labor lies in two distinct situations: the creation and the sustenance of the tool. Toolmaking is often an unsolicited activity. Tools prove their value during their lifecycle when it becomes visible that they are scientifically productive because they lead to significant findings, get cited in seminal papers, or because enough scholars use them in their research. Tools are born undesired by the public; they get appreciated after the fact. Tool creation is necessarily at the toolmakers' expense because there is no desire for it other than their own. Some may create tools as a hobby in their free time; many toolmakers are researchers who do it as part of their research, for instance, as part of a grant. In that sense, tool creation is not necessarily unfunded, but it may still be hidden insofar as the scientific institution does not differentiate it from other scientific activities like conducting fieldwork, extracting data, computing benchmarks, or writing papers. This camouflage may initially benefit the toolmaker because it protects their freedom; but if the tool gains a public life, problems arise.

Sustaining an existing tool entails a very different kind of hidden labor. Academic institutions typically face three difficulties when it comes to acknowledging tool sustenance: they ignore what it consists of, they do not expect it to be that way, and it does not fit their perspective on science.

First, universities typically ignore what kind of work is necessary to keep a tool functional. There is, of course, maintenance, invisible because it does not make an event. But the tool is not just the source code; it is also a number of artifacts necessary to its user community. The tool's life requires project and product management, technical watch, documentation writing, web design, and community management. The more alien those tasks are to the toolmaker's upper management, the more they may be considered unnecessary or even illegitimate.

Second, toolmaking is foreign to the university culture. Even when their management is supportive of the toolmaker's work, many practical problems arise. In their experience of maintaining DH projects at the KDL, Smithies et al. remark that often, "little support is offered from the surrounding culture. Funding agencies might require data management plans to ensure content is gracefully handled, depending on the country of origin, but appear unable to deal with the complex issues associated with the systems that generate and store that data. Collaborators often become uneasy at the use of 'industry' frameworks and 'business' language, suggesting (understandably) that it detracts from academic research culture. Meanwhile, some critics of the digital humanities appear to be unaware that a universe of very high quality, bespoke, but at-risk digital scholarship exists far away from the values and commercial imperatives of Silicon Valley ideology" (Smithies et al., 2019). This cultural resistance creates additional friction when the toolmakers' work is made visible. The hidden labor problem is not conjunctural but systemic.

The major point of systemic friction is the university's expectation that SSH outputs have no operating costs. Dissemination is supposed to cost something once, like article processing charges, but not all year long. Academic funding structures are unsuitable for fixed costs, which are usually handled by support functions. As Smithies et al. report about the KDL's situation, "the funders themselves signalled that they did not expect (or were not prepared to support) the development of long-term or permanent digital resources: without the goodwill of colleagues and the host institution most of them would have been closed years ago. Their future was often only discussed tangentially, elided in conversations between technical teams and PIs during the development process, in the optimistic hope 'something' would happen

eventually … PIs shifted emotional responsibility onto technical teams, and vice-versa: actual contractual responsibility was normally left undefined" (Smithies et al., 2019). An academic institution may have no interest in actively supporting tools. And when it does, it always means one of two things: dedicated work time from support staff, or one-time grants; it never means paid time for the researcher-toolmaker. Universities simply do not know how to evaluate tool maintenance by a researcher or, conversely, academic work done by an IT engineer. Which brings us to our last difficulty.

Third, toolmaking does not fit into universities' perspective on science, because it bridges the gap between support and academic functions (as we discussed in Chapter 21). This presents a risk for the toolmaker. In a situation where the university management comes to understand what toolmaking entails (difficulty 1) and acknowledges cultural frictions (difficulty 2), they may well conclude that toolmaking is not a legitimate scientific practice. Universities tend to think of tools as infrastructures, often drawing inspiration from famous natural science projects like the Large Hadron Collider: top-down endeavors maintained by a dedicated support staff but used by a separate group of academics and driven by a scientific board. Understandably, when a university pays to sustain a research infrastructure, it retains oversight in return. From the university's standpoint, sustaining a researcher's tool also presents risks: it may create a precedent (what if other researchers demand operating costs for their own scientific outputs?), and it engages the university's name without sufficient oversight, for instance, if the tool is involved in a data leak or illegal activities (public relations crisis). The academic freedom protecting the researcher-toolmaker also isolating him institutionally and tends to keep their work unaccounted for.

Toolmaking is a paradox for universities. They do not want support staff to have their own research agenda because this kind of agency is the privilege of academics. They will not let a research software engineer (RSE) create new research tools on their own, but they will let researchers do so insofar as it counts as a scientific output. Universities want tool creation to lie in the research side. However, they also want tool sustenance to lie on the support side. They want oversight and control over any project with operating costs while asking researchers to teach, publish, and bring new value in. Those conflicting demands make toolmaking difficult. Overcoming that mindset is possible, but it requires questioning a perspective deeply rooted in the culture of our scientific institutions.

We give Smithies et al. (2019) the last word on this topic. "Software sustainability will, in all likelihood, remain a pressing issue for the foreseeable future across all research disciplines. … Sustainable funding strategies need to be based on transparent costing that includes infrastructure and maintenance costs and made simpler and more reliable by established best practice. For this to be effective, realistic costing methods need to be developed and shared between product partners, and embedded within funding policy" (ibid, p. 13).

REPURPOSING PUBLICATION FOR TOOLMAKING

> At that point, [Stéfan Sinclar and I] had the academic problem of we have got to publish. So, to some extent, our solution was this praxis. We would plan a project and do it. We would, on the one hand, update *Voyant*, and on the other hand we would write a paper that we would give at a conference. The papers became a book, and *Voyant* 2.0 was released. Our book came out in 2016, *Hermeneutica*. (Geoffrey Rockwell, in Jacomy, 2023)

Although software development does not naturally pass as academic, toolmakers have multiple strategies to valorize it academically anyway. Researchers-toolmakers in particular, i.e. those who have an academic position, have an existential necessity to do so. This requires overcoming a problem of format. As Jay et al. (2021) remark, "whilst software is now central to the production of research, it is difficult—arguably impossible—to represent it adequately in standard scholarly publications". But toolmakers have found strategies. From the most covert to the most overt, they may (1) capitalize on tool-enabled science while keeping the tool itself secret; (2) release the tool publicly but only publish the science it enables; (3) publish a proxy paper; (4) publish the tool itself as scientific work. It is no mystery that the research community is evolving on that matter, and that overt strategies have become more viable in recent years. Let us take a tour of those strategies.

The first strategy is that of the closet toolmaker: a researcher who makes tools but does not want it to be known. In some sense, we could discuss whether their technology deserves to be called a tool (and the researcher themself a toolmaker) if it is not meant to be shared; but those demarcations are pointless. The point is that the closet toolmaker's strategy is to capitalize on the science enabled by their tool. Their tool is an asset to compete in the publication market, which does not incentivize them to share it publicly.

A variant of this situation consists of tools whose existence is known but that are so arcane to master that only a small group of people, typically around the toolmaker's laboratory, have the competencies to do so. The tools developed by Francis Chateauraynaud and Jean-Pierre Charriau, *Prospéro* and *Marlowe*, might be of that kind (Chateauraynaud, 2003; Chateauraynaud, 2011; for an overview, see Angermuller, 2006). The notoriety of these tools is undoubtedly centered on France, but even within French research, few have been able to use them for their own research. It might be more fair to see *Prospéro* and *Marlowe* as ongoing experiments, embodiments of a research program rather than tools meant to be fully operational in other people's hands. As such, these pieces of technology have been successful on a national level, if not beyond.

A second strategy consists of releasing a tool but only publishing the research it enables. This might be the most sensible thing to do if the tool has actually been developed for the sole purpose of supporting a case study. At the moment of publication, the tool might be seen as an appendix to the publication. However, if the tool is reused for other studies in the hands of other people, it may gain its autonomy. Yet the toolmaker may never publish the tool itself as a citable object. The original case study will often become what other researchers cite to pay their debt to the tool. This may well be a conscious strategy for the toolmaker. The tool *Contropedia*, for instance, was developed first for a case study (Weltevrede and Borra, 2016) which has become how it is mainly cited. In particular, for a researcher-toolmaker in the DCSSH, it is often easier to publish a case study than a tool paper because the publication venues for the former are more accessible than the latter, as we will now see.

A third strategy is the proxy paper: a publication is cited as a substitute for the tool itself. *Gephi* is a typical example. It is almost universally cited as Bastian et al. (2009), a simple poster presented at the ICWSM 2009 conference. That reference has received about 12,800 citations as we write (Google Scholar, August 2024) and received the ICWSM 2019 Test of Time Award, even though the content of the poster itself is barely relevant, if only because *Gephi* has considerably changed in a decade. Obtaining a peer-reviewed embodiment of the tool to receive citations was one of the goals of writing that poster. The title of a proxy paper will often start with the name of the tool, for instance "Scikit-learn: Machine Learning in

Python" (Pedregosa et al., 2011). It notably facilitates exposing the paper to the researchers searching for the tool in Google Scholar or another database. The toolmakers will often disclose how they would prefer the tool to be cited. The makers of *Scikit-Learn* explicitly point to the paper above on their website (SciKit-Learn team, 2024).

A case-study paper may also become the proxy paper, as Jay et al. report: "Most publication venues lack guidelines that encourage citing software directly … A common workaround is to cite a related paper instead. This might be a paper describing a larger study, where the software was integral to that research and is described in the methods section" (Jay et al., 2021).

Importantly, the proxy paper is not peer-reviewed as if it were the tool, but on its own merit. For instance, if the proxy paper is a case study, it is that case study that was evaluated, not the tool. For the toolmaker, this cuts both ways. On the one hand, it is possible to pass peer review with a good paper but a bad tool, for instance, with bugs or methodological issues, as long as it does not show in the paper. On the other hand, the toolmaker may have a solid tool but struggle to pass peer review, either because they are not that good at writing papers or because the journal's criteria are not adapted for a tool. A good example of this is the popular layout algorithm of *Gephi*: ForceAtlas2. A first version of that algorithm had been in *Gephi* for a few years, and users demanded a reference to cite. The algorithm designers also considered it worthy to document the algorithm and its method. They wrote a paper comparing the algorithm to the state of the art, disclosing the equations at stake, and accounting quantitatively for the different settings available, and submitted it to the open access journal *PLoS One*. The peer reviewer considered that the submission was not novel enough. The qualitative differences with other algorithms, in particular, did not pass as novelty. In fact, it was quite clear what the reviewers had in mind: quantitative evidence that the algorithm was more efficient. It turns out that the toolmakers could provide such a benchmark. They added it to the manuscript, and it was accepted (Jacomy et al., 2014). But it is pretty clear that this benchmark is irrelevant to the researchers who cite the paper.

The proxy paper has an obvious flaw: the tool evolves while the proxy paper remains the same. If the goal is to collect academic capital, then the tool passing into other hands raises an issue because the new authorship of the tool is no longer reflected in the proxy paper. Worse, proxy papers tend to reinforce their status over time because of their high citation count. If the tool is popular, there is a demand for citing the tool, and if the toolmakers do not fulfill that demand, someone else may do it in their stead. Perhaps less dramatic but more common, some tools get stuck with a proxy paper they have not completely chosen. For example, the most cited paper about *Jupyter Notebooks* is Kluyver et al. (2016), with about 3,200 citations (Google Scholar, August 2024). It is, de facto, a proxy paper. Yet in 2016, when someone asked, on the tool's GitHub, how to cite it, the toolmakers answered: "We're working on a proper citable reference paper for Jupyter, but that's taking some time - if you need something now, I think this paper in the proceedings of this year's ElPub conference is the most general citation we've got at the moment: [link to Kluyver et al., 2016]" (Jupyter, 2016). This GitHub issue (and the discussion) is still open at the moment of writing (August 2024). The 2016 paper emerged as the tool's proxy under the pressure of the researchers' need to pay their academic debt to it.

Fourth, the last and latest strategy: citing the tool directly, as an academic artifact. Let us start by clarifying that citing a tool is absolutely possible, aside from technical difficulties such as determining who the authors are or when the version was released. Formatting a reference for Gephi's current version at the moment of writing under the APA rules is possible: "Ramos

Ibáñez, E., Bastian, M., & Jacomy, M. (2023). Gephi (0.10.1) [Software]". The release cited here is not technically peer-reviewed, while the proxy paper is, but it does not prevent citing it, although some journals have the policy that references to non-peer-reviewed publications must be noted as such. Toolmakers may facilitate the process by associating the tool with a Digital Object Identifier (DOI), but a DOI is also not necessary to make something citable (Boettiger, 2013). To alleviate the technical difficulty further and/or obtain peer review, toolmakers now have the opportunity to publish what Jay et al. call a "software paper". It differs from a proxy paper in that peer review evaluates the tool itself, and not something else like a case study. The software paper "exists solely to describe the software, in a venue such as SoftwareX, the Journal of Open Research Software (JORS), the Journal of Open Source Software (JOSS), or F1000 Research" (Jay et al., 2021). Although it does not solve every issue, it has the benefit of being more true to the practice of toolmaking and more accessible to toolmakers who struggle to write academic papers.

The researcher-toolmaker who can navigate academia has access to all those strategies and can pick the most adapted in different situations. A very popular tool can also bring a number of citations well beyond what DCSSH scholars usually obtain. Toolmaking may also have some upsides to academic careers in the DCSSH if the toolmaker has enough success and the right strategies; but those strategies require the competencies to write and pass peer review efficiently enough. They are not accessible to less research-prone profiles like the research software engineer (RSE) who must instead rely on other ways, like partnering with a researcher, to be valued in academia.

> That is, I think, one of the genius things that we did: by developing this praxis of experiments, we were able to make the development academic. We were able to make sure that we got academic credit for the tool because we were always developing it while writing papers, delivering conference papers, and eventually the book, MIT press … We always made sure that every summer we had two or three papers at conferences. The papers reported on the tools and often had an academic component. But then we would say that we are also adapting this tool. … I think everyone in the digital humanities has to find a way [to make the development academic]. My colleagues in computer science have some of the same problems. Nobody is going to give them tenure for writing code. On the other hand, they will not give them tenure just for writing theory. (Geoffrey Rockwell, in Jacomy, 2023)

REPURPOSING GRANTS FOR TOOLMAKING

> One of the main things I do [to get funding] works because *Voyant* is sufficiently successful. Let us say that you were writing a grant. It is a big grant, and there is a place for *Voyant*, perhaps for a different tool and *Voyant*. We do a sort of deal. You write *Voyant* into your grant, and I write a letter of support; or you include me as a co-investigator, but in turn you put a budget line in there. So for five years, there is so much funding that comes to me to support *Voyant*, to do what you want to do, for example to add parts-of-speech tagging. (Geoffrey Rockwell, in Jacomy, 2023)

Obtaining grants is a valid strategy to fund DC tools for SSH, insofar as some conditions are met. The first condition is that the tool must be popular and renowned enough for a funding body to trust its academic validity. It must be clear that advancing the tool contributes to the scholarly project defended in the application. The second condition is that the application

must frame the tool support as creating academic value. In short, grants are not supposed to fund maintenance; the application must offer the implementation of new features. The third condition is that there must be someone to write the application. This sounds mundane, but it might be the biggest issue. It is complicated for a single person to have the necessary time and competencies to do it all: code the tool, write papers (and teach), and write grant applications. More realistically, the toolmaker is part of a collaborative network that helps them with that, either because better developers support the tool while they manage the project (notably by writing grants), or because some better researchers include them in their applications so that they can focus on the tool support. In both cases, the toolmaker needs a support network.

Let us spell it out: as long as the maintenance of DC tools for SSH is taboo, toolmaking is unsustainable in these disciplines. Unless it gets funded explicitly, the two only ways to get maintenance done are either on the toolmakers' personal time or camouflaged as something else, like adding features or using tools for studies. It is actually quite easy for the toolmaker to overcharge the cost of implementing a feature in a grant application so that they get some time to do maintenance. To some extent, all the parties involved may even be aware of it and implicitly agree with the practice. This, from a practical standpoint, may look like a win-win situation; but it does not solve the problem of sustaining the tool over the long term. In fact, it makes the problem worse because it reinforces the taboo. Smuggling toolmaking as if it were something else has been the norm for a long time, but the situation is changing, and new ways to make it sustainable are emerging.

THE SITUATION OF DCSSH TOOLMAKING IS CHANGING

As the practice of coding normalizes, it becomes better accepted as scholarly production. The commoditization of computational means, for instance through the use of Jupyter Notebooks, supports a more nuanced gradient of hybrid scholars. It is less and less necessary to commit to one of two sides, development or research. Many researchers engage with computational methods without defining themselves as digital or computational scientists. The coding part is just a moment in the methodological journey prompted by their data and their research questions. As everyone gets to code a little bit, coding is acknowledged more often as a form of writing.

It has become easy to publish code, for instance via the Open Science Framework (Foster & Deardorff, 2017), or via a DOI from Zenodo (European Organization for Nuclear Research & OpenAIRE, 2013). Zenodo even allows versioning software to represent a new release and automatically updates the DOI on a GitHub commit. Furthermore, the *Journal of Open Research Software* (JORS) "features peer reviewed Software Metapapers describing research software with high reuse potential. We are working with a number of specialist and institutional repositories to ensure that the associated software is professionally archived, preserved, and is openly available. Equally importantly, the software and the papers will be citable, and reuse will be tracked" (Software Sustainability Institute, 2024).

In some countries, like the Netherlands (Quality and Relevance in Humanities, 2024), the quality and relevance indicators in the humanities now also include software (albeit often underspecified as to how it counts and without mention of software maintenance). In the United Kingdom, the KDL has been formalizing a Checklist for Digital Outputs Assessment (Ciula, 2019) and the Software Sustainability Institute has proposed a guide to good practice

in submitting software outputs (Jay et al., n.d.). In France, Gomez-Diaz and Recio (2019) have proposed the "CDUR" protocol for the evaluation of research software (RS), consisting of the chronological steps of "Citation, Dissemination, Use and Research" (ibid), addressing a series of identified problems: the software production is not registered as research production on its own (just as a means to a paper); the software has no clear status (name, authors, dates, version …); it is disseminated chaotically (on personal blogs, forges, social media … but not the lab's page); it has no license; its legal status is problematic because it has no license or authors; the laboratory has no declared policy on software production; and the software production is not evaluated. Of course, research tools generally don't have all of those problems. The CDUR protocol has been designed to address these issues in order and help not just the toolmaker but also the evaluation committees and decision makers to account for the value of research software production.

We have seen that opening the sources of a tool does not suffice to make it sustainable. Neither does a proxy paper, a Zenodo entry, or a grant application. However, many small changes in research practices may lead to shifts in academic cultures and transform institutions. The presence of research software engineers helps preserve technical competencies and normalize software practices in academia. Alleviating the maintenance taboo still requires the emergence of semi-permanent structural funding, but this might be forthcoming. In Europe, the Netherlands is particularly active on that front, for instance with the short-lived PDI-SSH infrastructure grant that funded the CAT4SMR project (DMI-TCAT, YouTube Data Tools, 4CAT). Also noteworthy is the Amsterdam Declaration on Funding Research Software Sustainability, which aims to change how funders approach research software. Furthermore, the convergence and consolidation of the open source and open science movements may give rise to the institutional will, at the European level, to build sustainable digital commons. For instance, the Dutch NLnet Foundation has been remarkably successful at stabilizing small- and medium-scale open source projects.

Tools and toolmakers are of many kinds and in various situations, and no single model will work for everyone, but even if we cannot foresee where future breakthroughs improving the toolmaker's condition may happen, trends seem to give us reasons to be optimistic—for the toolmaker and for the research community in general. As Jay et al. conclude in their quasi-manifesto, "official recognition of software as a research output will ultimately be transformative, improving the quality, reproducibility and scalability of our knowledge production, as well as recognising the often hidden role of the increasing number of scholarly researchers who spend most of their time writing code" (Jay et al., 2021).

REFERENCES

Angermuller, J. (2006). Francis Chateauraynaud, Prospéro. Une technologie littéraire pour les sciences humaines. *Mots. Les langages du politique*, (81), 119–121.

Bastian, M., Heymann, S., & Jacomy, M. (2009). Gephi: An open source software for exploring and manipulating networks. In *Proceedings of the International Aaai Conference on Web and Social Media*, *3*(1), 361–362.

Boettiger, C. (2013). Doi != Citable. *Lab Notebook*, June 3. https://www.carlboettiger.info/2013/06/03/DOI-citable.html (accessed 19/06/2024).

Chateauraynaud, F. (2003). Marlowe. Vers un générateur d'expériences de pensée sur des dossiers complexes. *Bulletin de méthodologie sociologique/Bulletin of sociological methodology*, (79), 6–32.

Chateauraynaud, F. (2011). Argumenter dans un champ de forces. Essai de balistique sociologique. *Sociologie Du Travail, 55*(2), 270–271. Éditions Petra.

Ciula, A. (2019). KDL Checklist for Digital Outputs Assessment. https://doi.org/10.5281/zenodo.3361580

Derrick, G., Hettrick, S., Baker, J., Karoune, E., Kerridge, S., Fletcher, G., Chue Hong, N., Ballantyne, L., Fransmann, J., & Roche, T. (2024). Shaping the future of research evaluation: Insights from The Festival of Hidden REF. https://doi.org/10.1108/EGP-02-2024-0001

European Organization For Nuclear Research, & OpenAIRE (2013). *Zenodo.* https://doi.org/10.25495/7GXK-RD71

Foster, E. D., & Deardorff, A. (2017). Open science framework (OSF). *Journal of the Medical Library Association*, *105*(2), 203.

Gomez-Diaz, T., & Recio, T. (2019). On the evaluation of research software: the CDUR procedure. *F1000Research*, *8*, 1353.

Jacomy, M. (2023). Interview of Geoffrey Rockwell, maker of Voyant Tools. *Reticular.* https://reticular.hypotheses.org/8039

Jacomy, M., Venturini, T., Heymann, S., & Bastian, M. (2014). ForceAtlas2, a continuous graph layout algorithm for handy network visualization designed for the Gephi software. *PloS one*, *9*(6), e98679.

Jay, C., Haines, R., & Katz, D. S. (2021). Software must be recognised as an important output of scholarly research. *International Journal of Digital Curation*, *16*(1), 6. https://doi.org/10.2218/ijdc.v16i1.745

Jay, C., Hettrick, S., Chue Hong, N., Haines, R., & Grieve, S. (n.d.). *Good Practice in Submitting Software Outputs to REF2021.* Software Sustainability Institute. https://www.software.ac.uk/guide/good-practice-submitting-software-outputs-ref2021

Software Sustainability Institute (2024). *Journal of Open Research Software*. https://openresearchsoftware.metajnl.com/ (accessed 03/09/2024)

Jupyter (2016). *Citing Jupyter · Issue #190 ·* jupyter/jupyter. GitHub. https://github.com/jupyter/jupyter/issues/190 (accessed 02/09/2024)

Kerridge, S., Poli, S., & Yang-Yoshihara, M. (Eds.) (2023). *The Emerald handbook of research management and administration around the world.* Emerald Publishing Limited.

Kluyver, T., Ragan-Kelley, B., Pérez, F., Granger, B., Bussonnier, M., Frederic, J., Kelley, K., Hamrick, J., Grout, J., Corlay, S., Ivanov, P., Avila, D., Abdalla, S., & Willing, C. (2016). Jupyter Notebooks–a publishing format for reproducible computational workflows. In B. Schmidt & F. Loizides (Eds.), *Positioning and power in academic publishing: Players, agents and agendas* (pp. 87–90). IOS Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Quality and Relevance in Humanities (2024). *Software*. https://www.qrih.nl/en/case-studies-and-indicators/qrih-indicators/research-products-for-peers/software (accessed 03/09/2024).

Ramos Ibáñez, E., Bastian, M., & Jacomy, M. (2023). *Gephi* (0.10.1) [Software].

Schreibman, S., & Hanlon, A. M. (2010). Determining value for digital humanities tools: Report on a survey of tool developers. *Digital Humanities Quarterly*, *4*(2). https://www.proquest.com/docview/2555208512/abstract/8B9E2E7ABEE94C7EPQ/1 (accessed 01/09/2024).

SciKit-Learn team (2024). *About us*. Scikit-learn. https://scikit-learn.org/stable/about (accessed 02/09/2024).

Smithies, J., Westling, C., Sichani, A. M., Mellen, P., & Ciula, A. (2019). Managing 100 digital humanities projects: Digital scholarship and archiving in King's Digital Lab. *Digital Humanities Quarterly*, *13*(1).

Weltevrede, E., & Borra, E. (2016). Platform affordances and data practices: The value of dispute on Wikipedia. *Big Data & Society*, *3*(1). https://doi.org/10.1177/2053951716653418

25. In defense of the researcher-toolmaker figure

Mathieu Jacomy, Bernhard Rieder, Erik Borra, Guillaume Plique, Kenneth Christian Enevoldsen and Nees Jan van Eck

As a conclusion, we would like to circle back to the toolmaker as a figure in the research group. In Chapter 20, we contended that tools bear theory. They are instruments of power and contribute to the politics of method (Adams et al., 2005). This point also underscores the difference between the research software engineer (RSE) and the researcher-toolmaker regarding authorship, but in a different way. Ownership of the tool entails methodological accountability. The supposed agnosticism of the RSE incentivizes them to withdraw from authorship, retreat to the field of techno-methodological support, and dodge accountability. The RSE is in a weak position to defend or argue about the tool's scientific impact because it is the job of academics. The position of the researcher-toolmaker is stronger in that regard, not only because they are more prone to endorse authorship and accountability but also because they have a direct interest in defending the tool as an embodiment of their own research. Although, as we have seen, the tool is always more than its maker's intent and may empower users in unforeseen ways.

We have seen in Chapters 21 and 22 that toolmaking in the DCSSH covers a spectrum of profiles and situations, some leaning towards maintaining infrastructures, others leaning towards inventing science experiments. The toolmaker is generally proud of their tool and cares for it, even when it causes them trouble. This attachment is a necessary counterpoint to many burdens: maintenance and repair, fundraising, project management … Whether or not the tool is theirs makes a difference to the toolmaker, which in turn impacts the tool's design, presentation, and dissemination within the DCSSH community.

This is a first point where we find it useful to make the distinction between the RSE and its counterpart on the spectrum of toolmaking, the researcher-toolmaker (or "researcher-developer" for Baxter et al., 2012). The researcher-toolmaker has an academic position and the tool they make is an extension of their research, an expression of their scientific agenda. In contrast, the RSE is in a support position, framed by the institution as a resource for scholars whose scientific agenda supersedes theirs (if it even realizes that an RSE may have one). The logic of this division of labor calls for tools to be invented by researcher-toolmakers and then maintained by the RSE. But this omits that the former is attached to their tool and does not want to give up control and authorship, while the latter is not and may not want to assume control and authorship. The RSE profile is about infrastructure care but falls short regarding tools that do not look like infrastructures but rather products.

We overviewed the lifecycle of tools in Chapter 23, and we have seen that many DC tools for SSH are born from the practice of a researcher-toolmaker, but also that maintenance is a difficult burden for the toolmaker to face. In this regard, the researcher-toolmaker is in the weaker position. Maintenance is recurring but invisible, and as we developed in Chapter 24, it belongs to the regime of operations (seen as fixed costs), as opposed to projects (seen as investments). In the traditional division of labor in universities, operations are supposed to be

assigned to RSEs while project leadership goes to researchers. Here, again, it is useful to make the distinction between the RSE and the researcher-toolmaker. It is much easier for an RSE-toolmaker to justify repair and maintenance work, but because the RSE is supposed to support others, there is also a risk that their management assigns them to other tasks than maintaining their tool. The researcher-toolmaker has the opposite problem, where they are free to maintain their tool but must do so during research time that no funding body is willing to sponsor.

Similarly, caring for a tool entails project management labor, such as writing grants applications or communicating with contributors and users, which is better assumed by a researcher-toolmaker than an RSE.

Our last point in this piece will be to defend the figure of the researcher-toolmaker as an important persona to rethink the conditions of production of the DCSSH apparatus. In short, there is a risk that the emerging figure of the RSE leads to reinforcing an institutional divide that is antinomic to toolmaking as a practice. But let us preface our argument with the acknowledgment that we see the emergence of RSE networks as a fantastic advancement to the cause of toolmaking in academia. We also acknowledge that the RSE is not necessarily a toolmaker, and that caring for academic infrastructures is contributing to science on its own, regardless of whether or not the caretaker understands their practice as making tools.

Our observations lead us to believe that in reality, there is a much richer continuum of profiles than the support/academic divide suggests. Some RSEs publish and assert a scientific agenda as if they were academics; and some researcher-toolmakers have found strategies to get the maintenance labor done. But most often, the key to sustainable toolmaking is collaboration across the support/academic divide. We have seen this pattern at the Digital Methods Initiative in Amsterdam, at the Sciences Po médialab in Paris, and at the King's Data Lab in London (among other places). The ideal collaboration is between an RSE-toolmaker and a researcher-toolmaker, i.e. between a research-prone technician and a development-prone academic. Being present on both sides of the institutional divide but being interchangeable enough, they can find the best strategies to make their tool successful in the long run. It was the form of the mythical collaboration between Stéfan Sinclair and Geoffrey Rockwell on *Voyant*.

We defend the figure of the researcher-toolmaker because the prominence of the RSE should not hide that many tools we use in the DCSSH were invented by people with a scientific vision and a commitment to critical technical practice (Agre, 1997). As infrastructured theory, a tool is never the neutral embodiment of a pre-existing method, if only because the implementation reshapes the method. It enacts something new and often unexpected. But, as a matter of fact, the toolmaker never invents the method before the tool; rather, they develop the method as the tool. The toolmaker's methods are natively computational and only secondarily published material, regardless of the toolmaker's position.

The managerial delusion that our scientific apparatus can be imagined by academic brains and maintained by support hands goes against everything we know about toolmaking. We hope that upcoming institutional changes will acknowledge this reality to support more hybrid and flexible forms of scientific engagement with technology, on both sides of the support/academic divide.

REFERENCES

Adams, J., Keane, W., & Dutton, M. (2005). *The politics of method in the human sciences: Positivism and its epistemological others*. Duke University Press.

Agre, P. E. (1997). Toward a critical technical practice: Lessons learned in trying to reform AI. In G. Bowker, S. L. Star, L. Gasser, & W. Turner (Eds.), *Social science, technical systems, and cooperative work: Beyond the Great Divide* (pp. 131–157). Psychology Press.

Baxter, R., Hong, N. C., Gorissen, D., Hetherington, J., & Todorov, I. (2012). The research software engineer. *Digital Research 2012* (pp. 1–3). University of Edinburgh. https://www.pure.ed.ac.uk/ws/files/65195747/DR2012_12_1_.pdf